人工智能与地理数据挖掘

第十一讲　深度学习

孙伟 sunwei29@mail.sysu.edu.cn

地理科学与规划学院

中山大學
SUN YAT-SEN UNIVERSITY

2026.6

数据挖掘/机器学习的典型方法

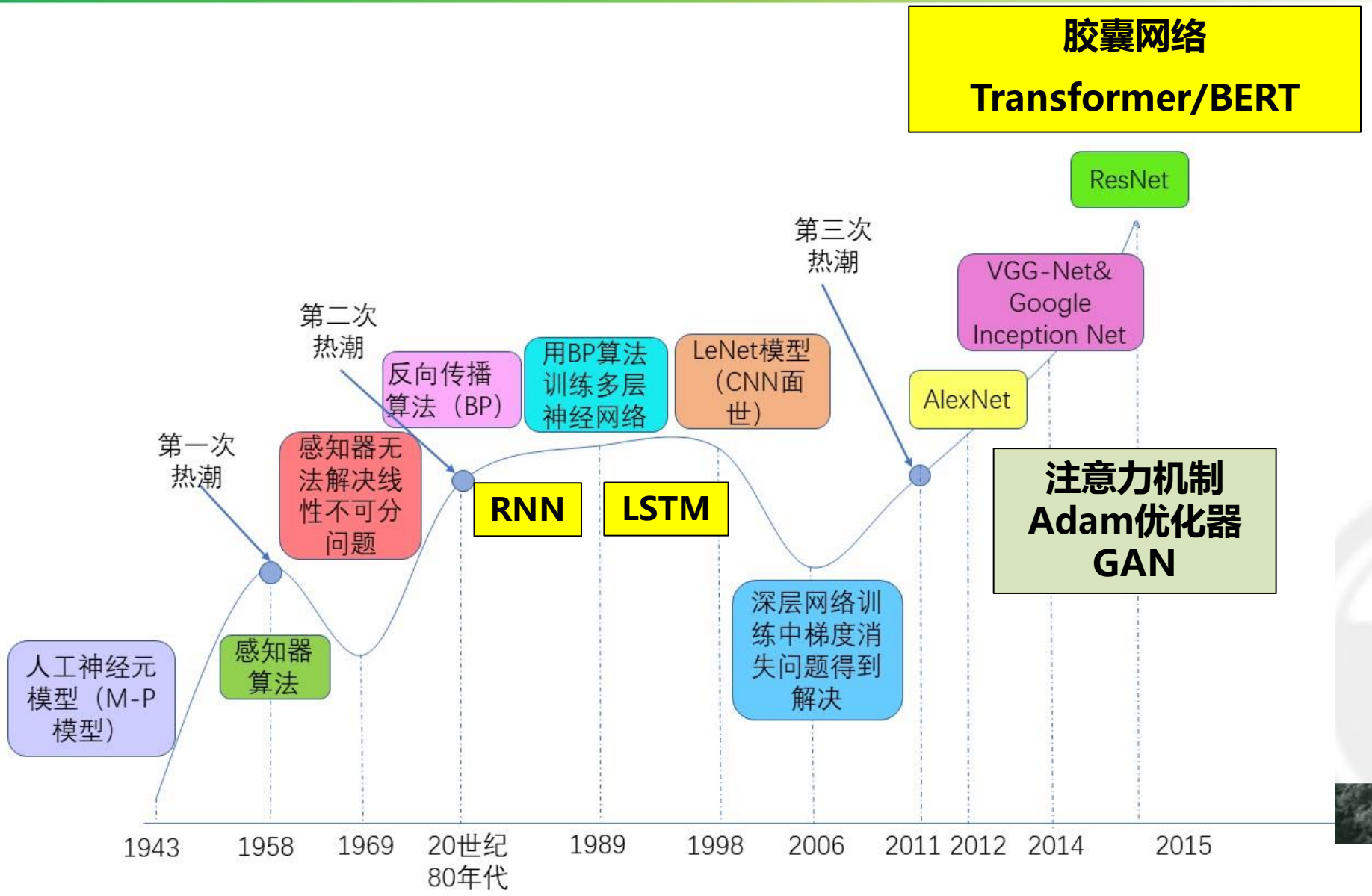
类别	聚类	降维	回归	分类
线性方法		相关性 主成分分析 因子分析	多元线性回归	Logistic回归 判别分析
最近邻	k均值聚类		kNN	kNN
树型	层次聚类		决策树：回归树	决策树：分类树
密度	DBSCAN			
神经网络	SOM		ANN	ANN
映射函数			SVM（SVR）	SVM（SVC）
图论	谱聚类		概率图模型	概率图模型
随机变量	高斯混合模型			贝叶斯分类
模糊数	模糊C均值聚类		模糊逻辑模型 ANFIS	模糊逻辑模型
深度学习			CNN，LSTM	CNN

主要内容

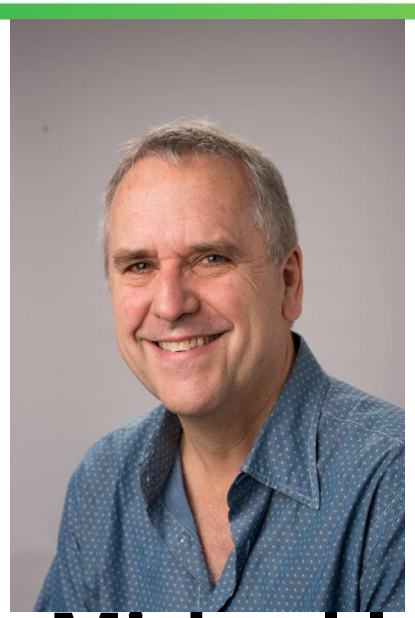
1. 深度学习的发展史和实例
2. 循环神经网络（RNN）模型
3. 长短期记忆网络（LSTM）模型
4. 卷积神经网络（CNN）模型
5. 用Matlab构建深度学习模型
6. 小结

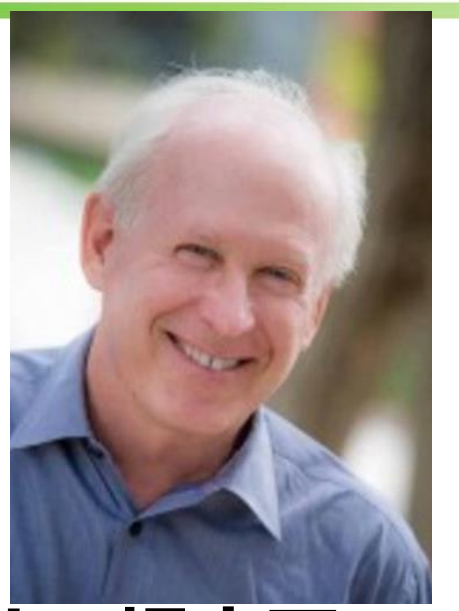
深度学习的发展史和实例

神经网络到深度学习的发展史1



1986, 1990 简单循环网络

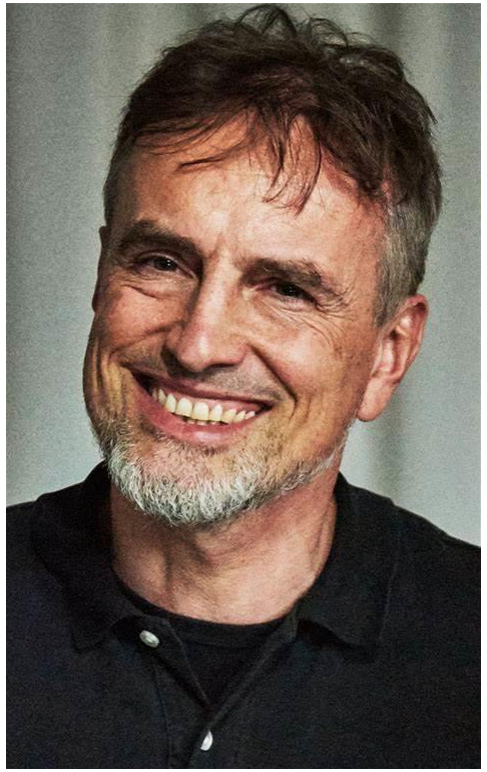




- 1986年，Michael I. Jordan提出了Jordan网络 。Jordan网络的每个隐含层节点都与一个状态单元相连以实现延时输入。
- 1990年，Jeffrey Elman提出了第一个全连接的RNN，即Elman网络 。
- Jordan和Elman网络都从单层前馈ANN出发构建递归连接，称为Simple Recurrent Network, SRN

1997 长短期记忆网络





Sepp Hochreiter和Jurgen Schmidhuber在1997年提出包含门控的**长短期记忆网络**（Long Short-Term Memory networks, LSTM），解决标准RNN时间维度的梯度消失问题。

2010年至今：自然语言处理模型

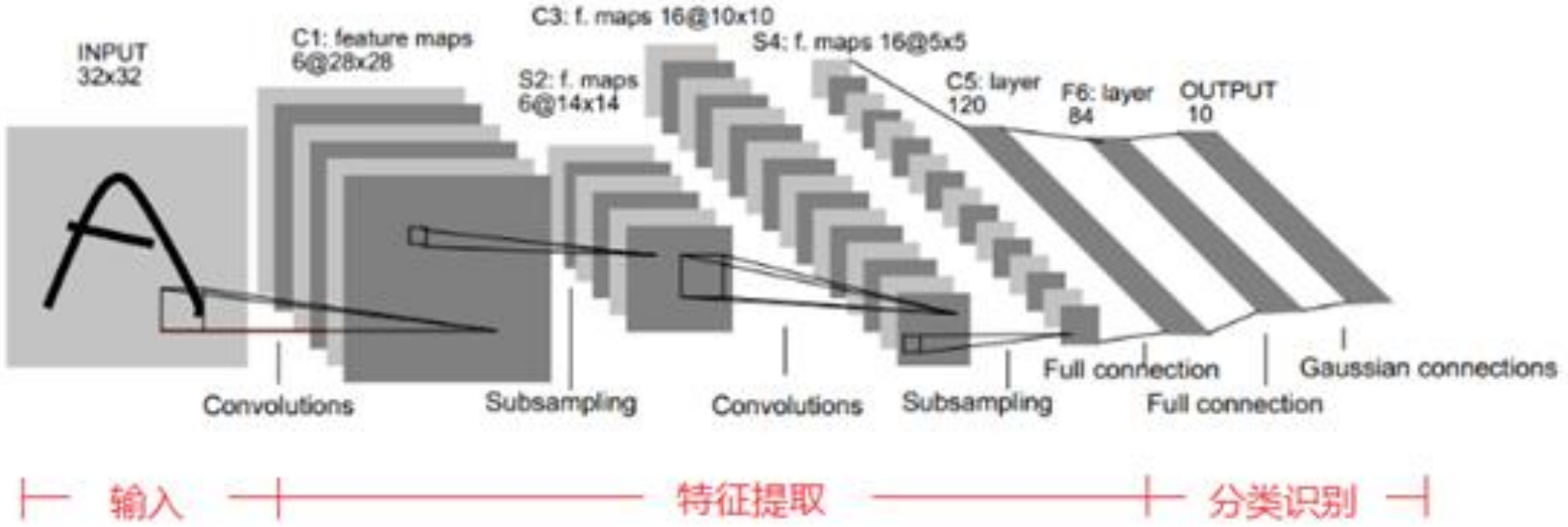
2014年，K. Cho提出了门控循环单元网络（Gated Recurrent Unit networks, GRU），该方法是LSTM之后另一个受到关注的RNN门控构架 。

2010年，Tomas Mikolov及其合作者提出了基于RNN的语言模型 。

2013-2015年， Y. Benjo、D. Bahdanau等提出了编码器-解码器、自注意力层等一系列RNN算法，并将其应用于机器翻译。

为语言模型设计的RNN算法在随后的研究中启发了包括Transformers、XLNet、ELMo、BERT等 。

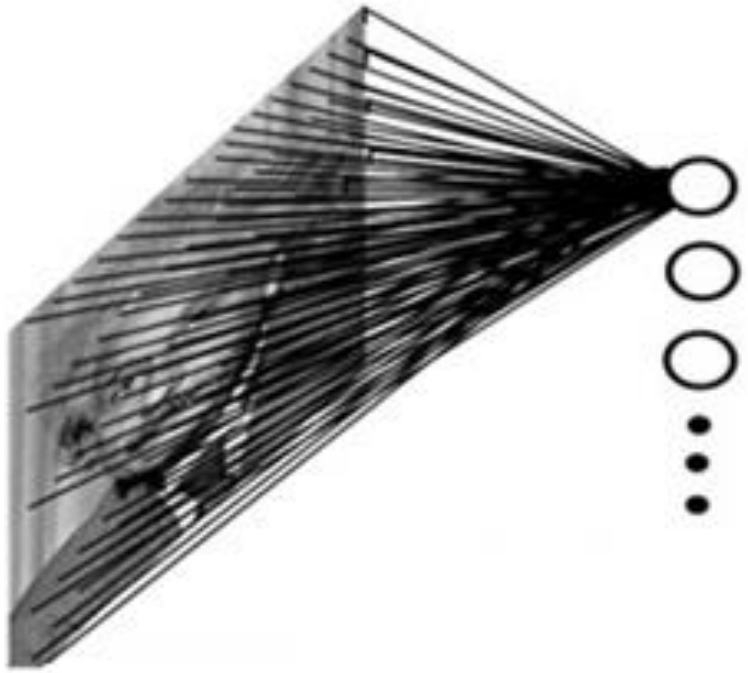
1994-1998 LeNet



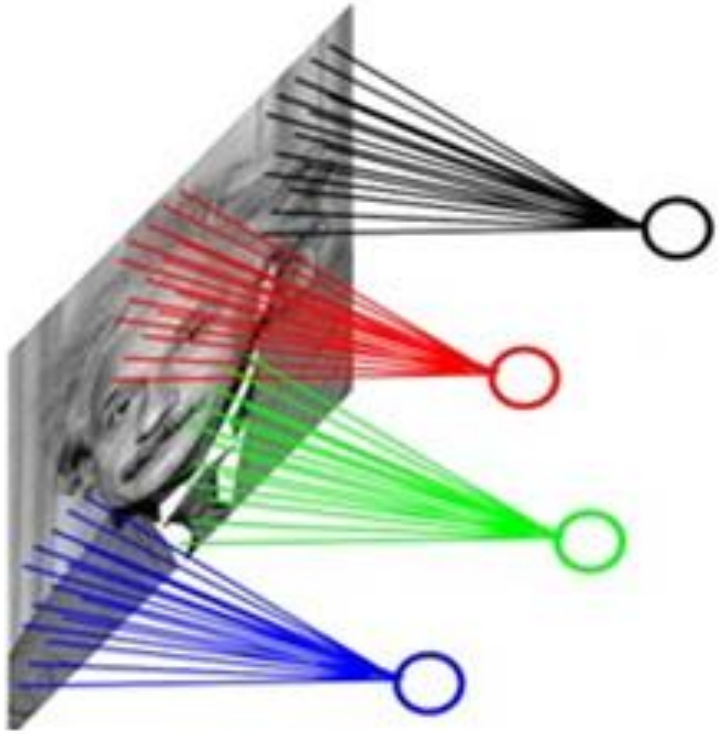
- Yann LeCun 1998年发表LeNet的经典论文《Gradient-Based Learning Applied to Document Recognition》
- LeNet5诞生于1994年，是最早的CNN之一，利用卷积、参数共享、池化等操作提取特征，避免了大量的计算成本，最后再使用全连接ANN进行分类识别。

全连接到局部连接

全连接模式（经典神经网络）

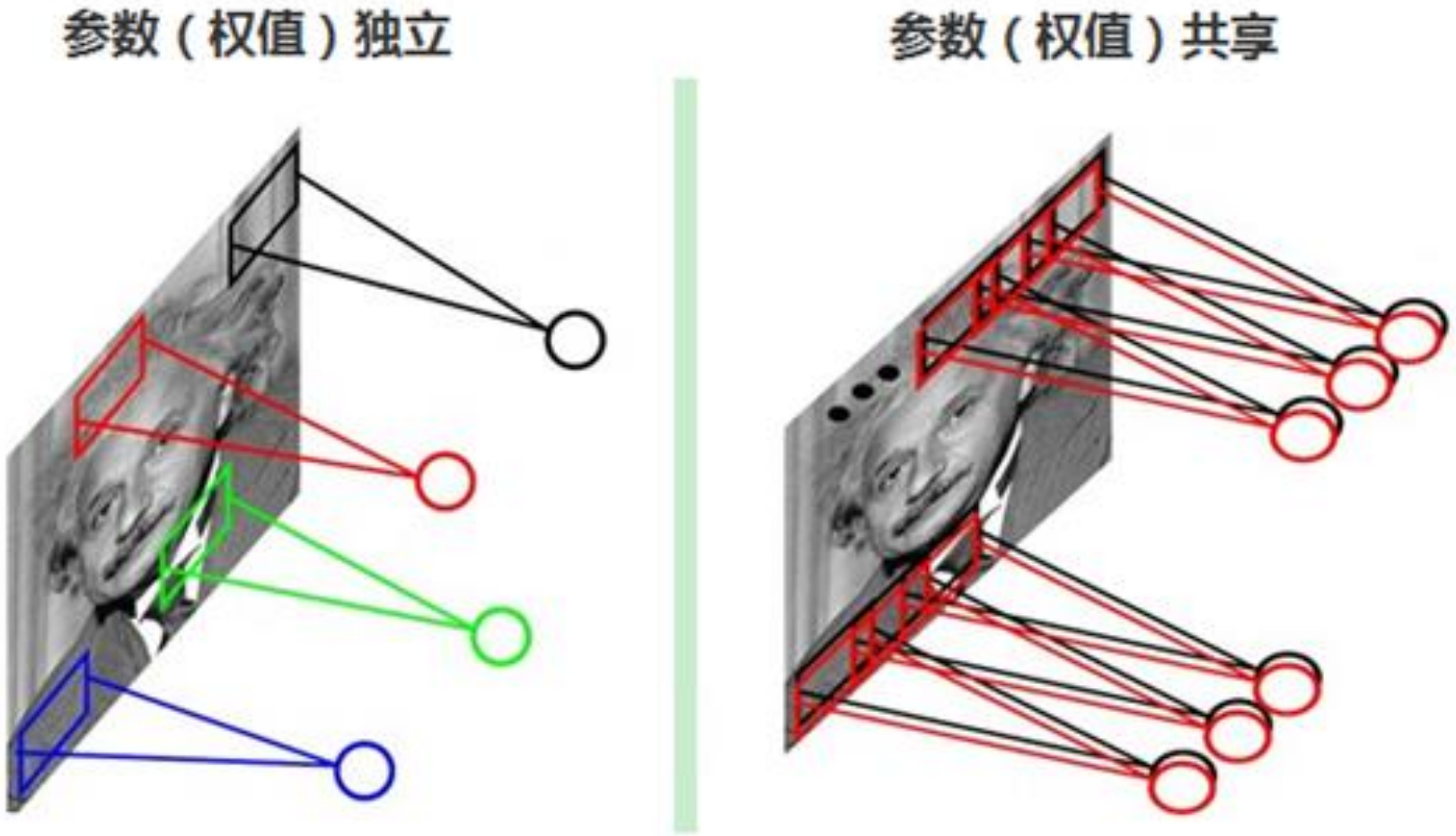


局部连接模式（卷积神经网络）

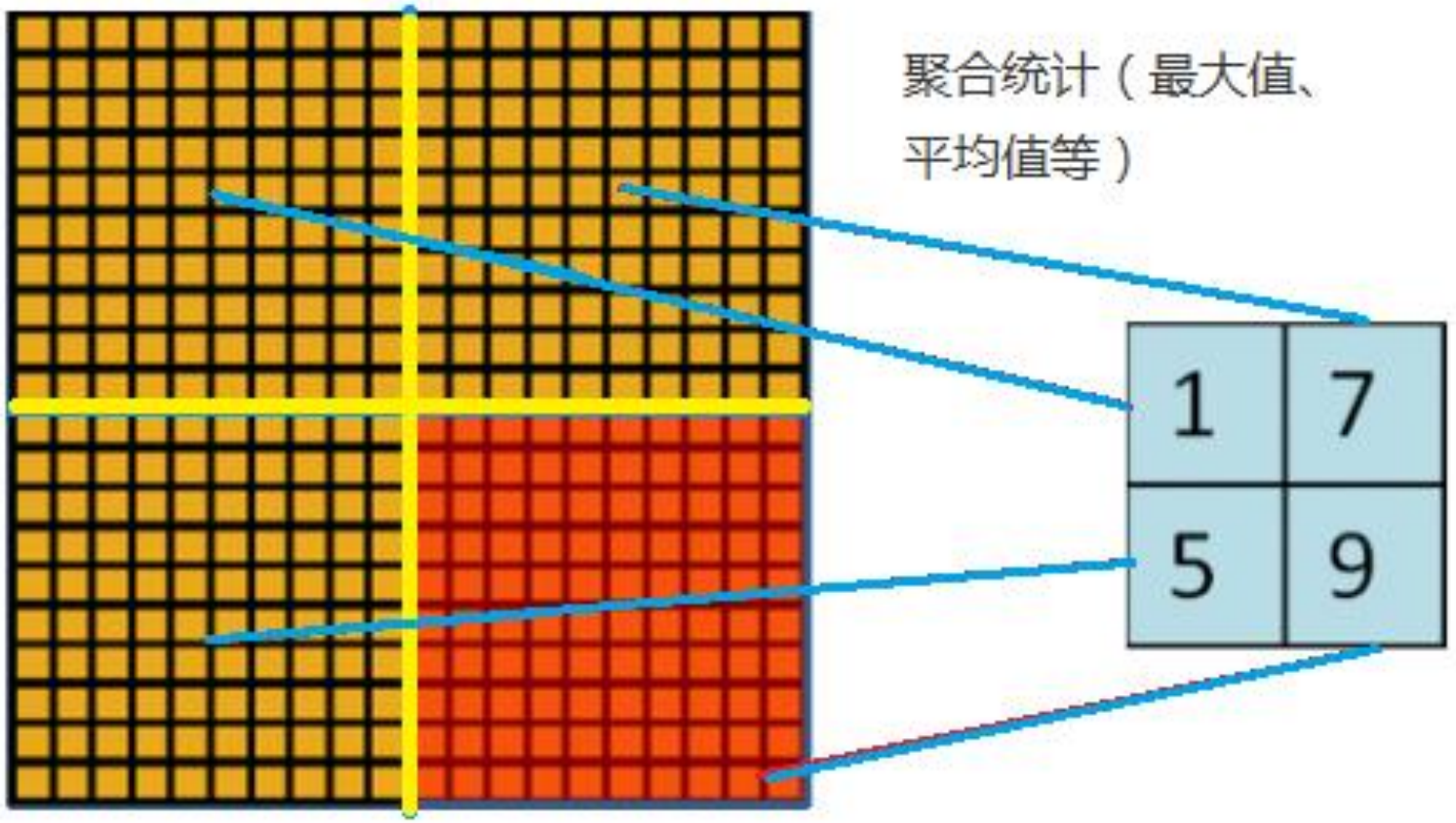


局部感受野

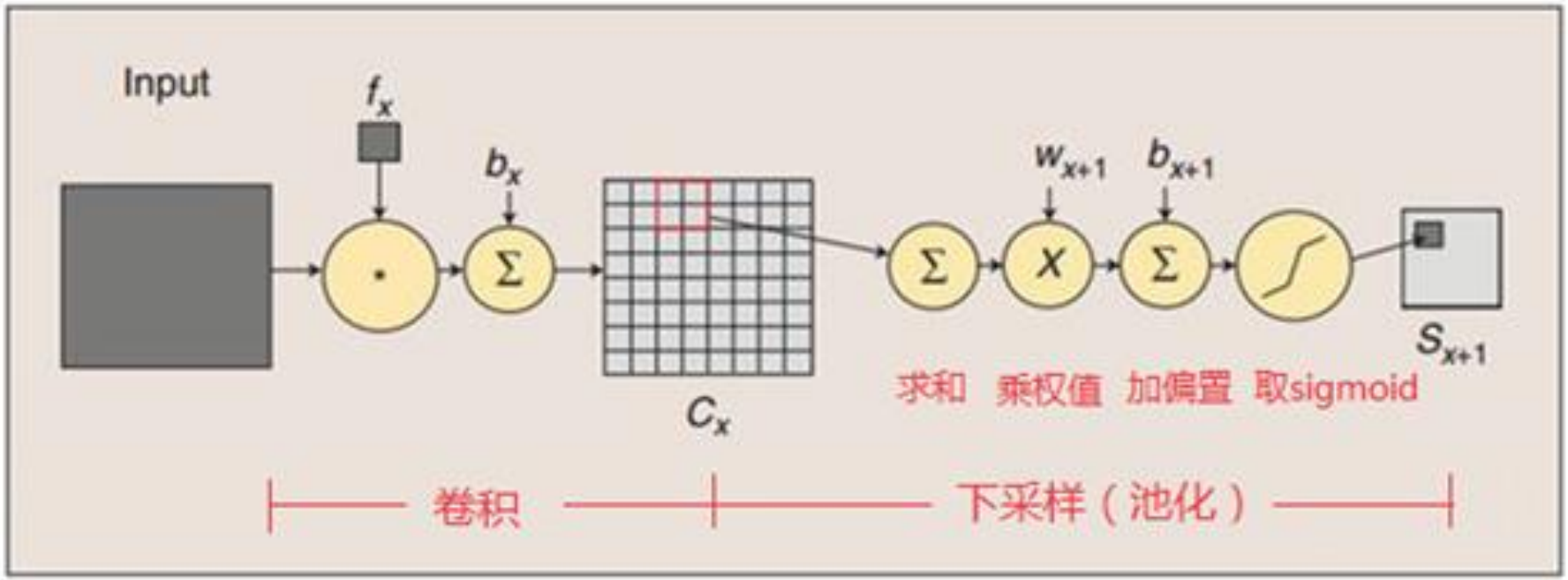
参数（权值）共享



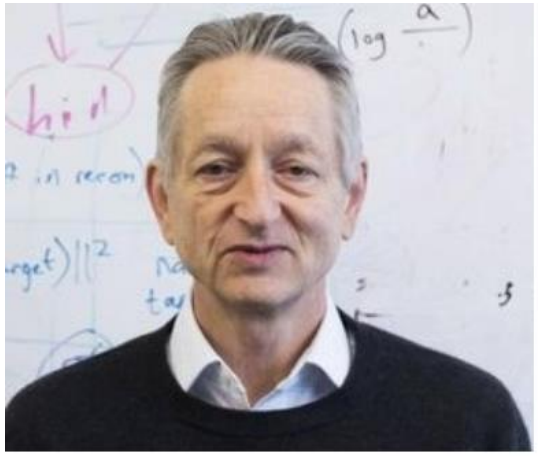
池化（特征映射/特征降维）



卷积+池化



2003年 深度学习三剑客

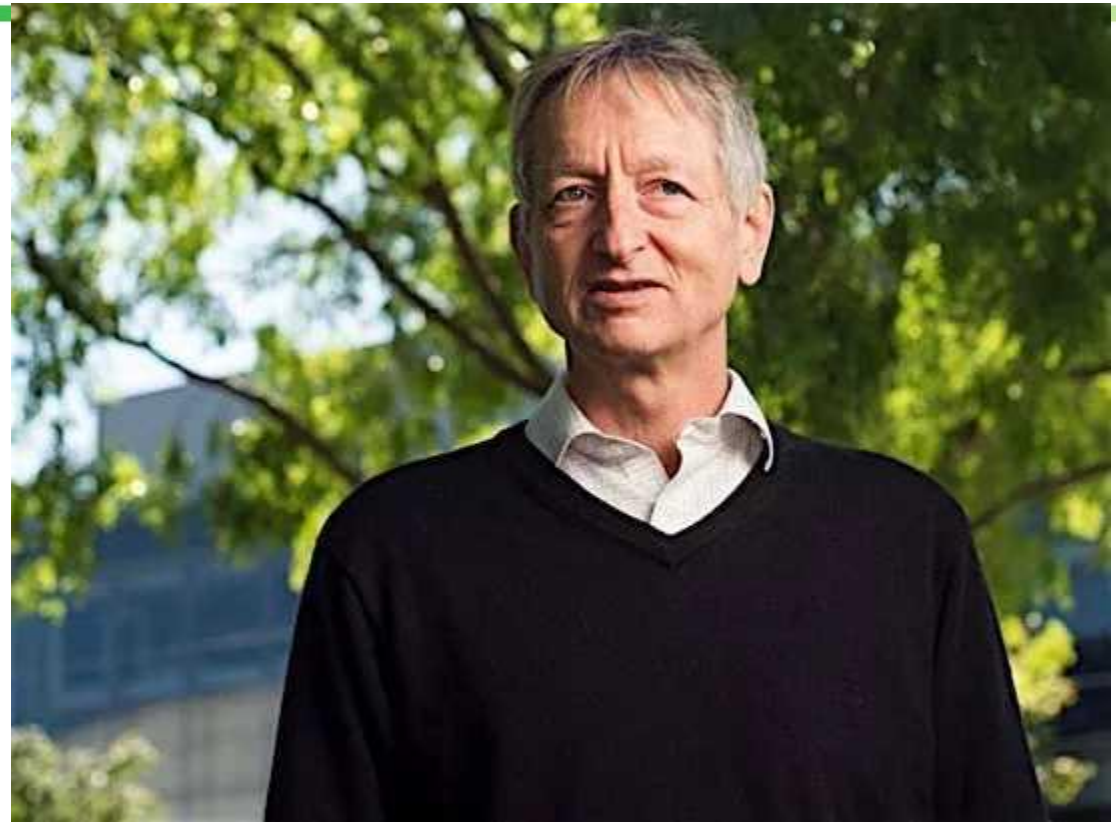

Geoffrey Hinton


Yann LeCun


Yoshua Bengio

- **LeCun**:“这种算法很有价值，不知为什么要放弃它。”
- **Hinton**: “智能产生于人脑，所以从长远来说，人工智能应该像大脑系统一样工作。”
- 三人从2003年开始，组成了“Deep learning conspiracy”,悄悄开发了10层以上的ANN。

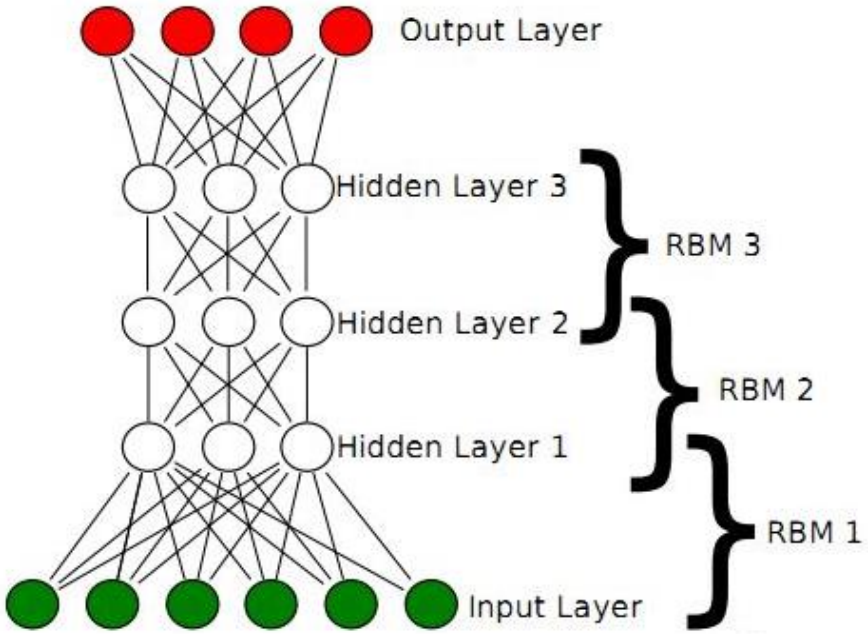
2006深度信念网络(DBN)

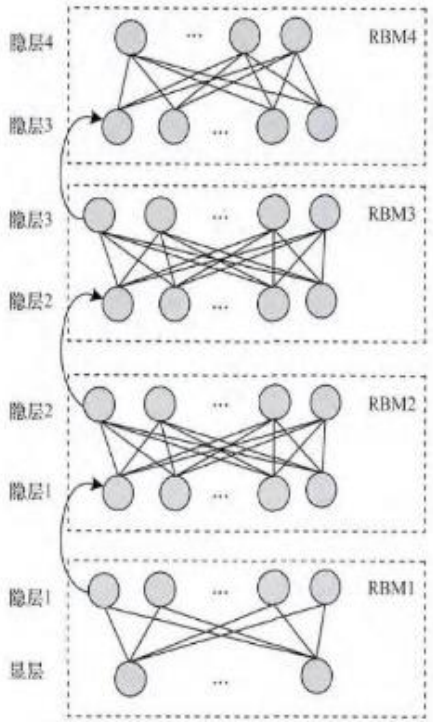


2006年, Hinton提出深度信念网络(DBN), 通过
“逐层预训练+微调”使得训练深度模型变得相对容易

https://my.oschina.net/u/876354/blog/1626639

逐层预训练





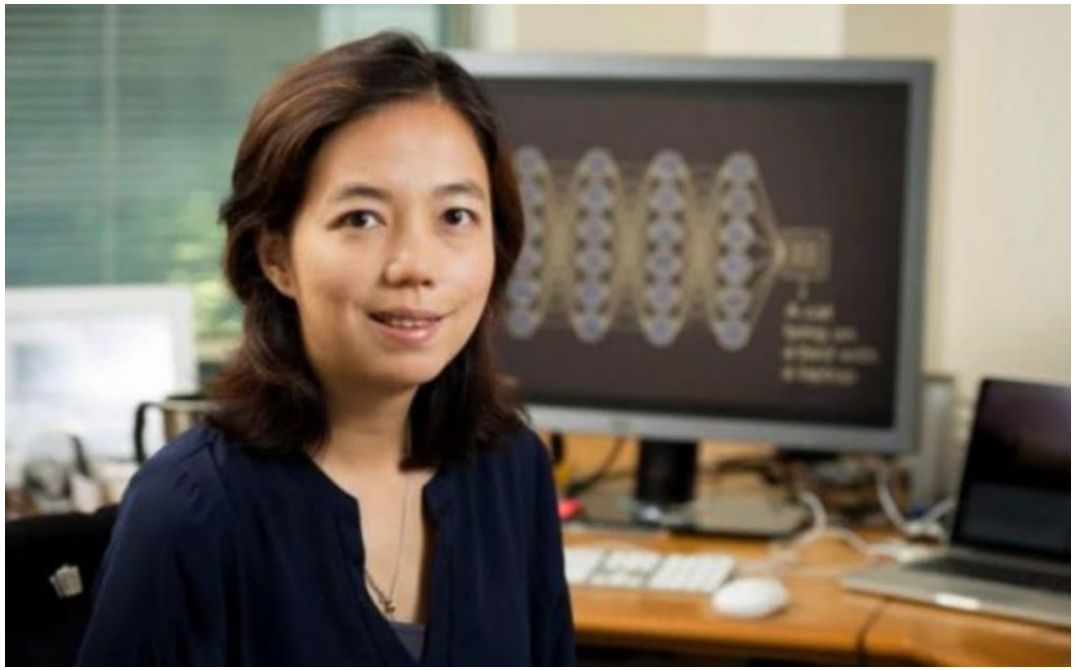
逐层的训练过程如下：

（1）最底部RBM以原始输入数据进行训练

（2）将底部RBM抽取特征作为顶部RBM的输入继续训练

（3）重复这个过程训练以尽可能多的RBM层

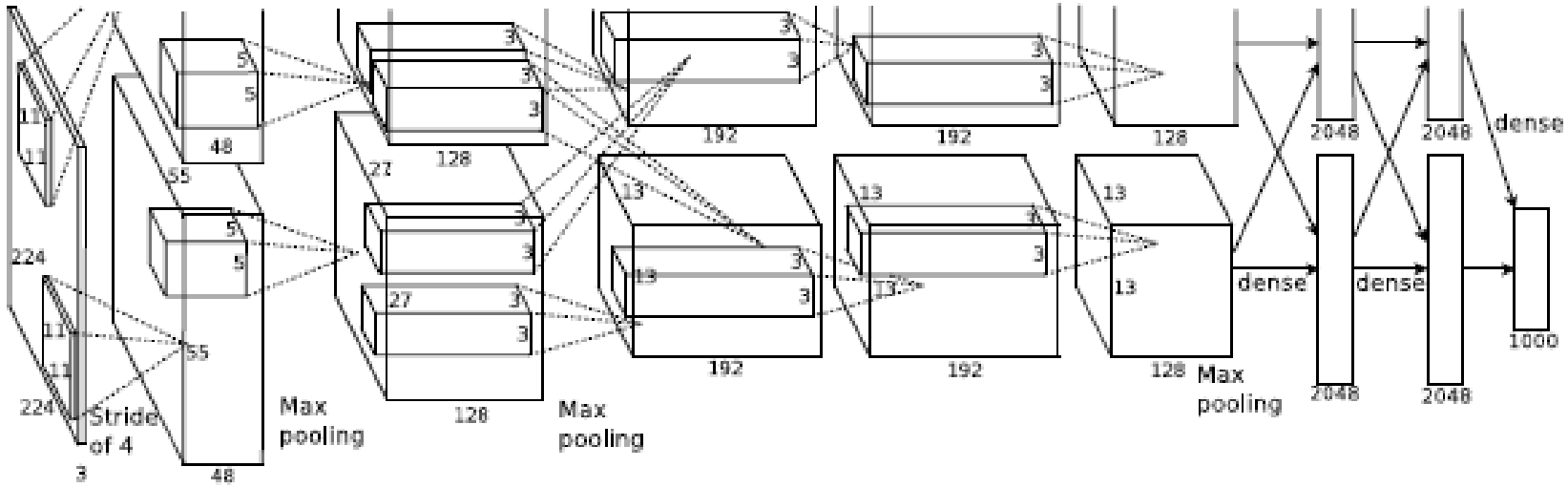
2009 发布ImageNet 数据库



2000年前后，计算机视觉一些模型的适用范围有限，李飞飞观察儿童学习后提出：**问题不在于模型而在于数据。**

2009 年，李飞飞团队用Amazon众包平台，收集了320 万张的（后增到 1500 万张）大型图片集，**发布ImageNet 数据库**，引入大规模视觉识别挑战ILSVRC。

2012 Alexnet

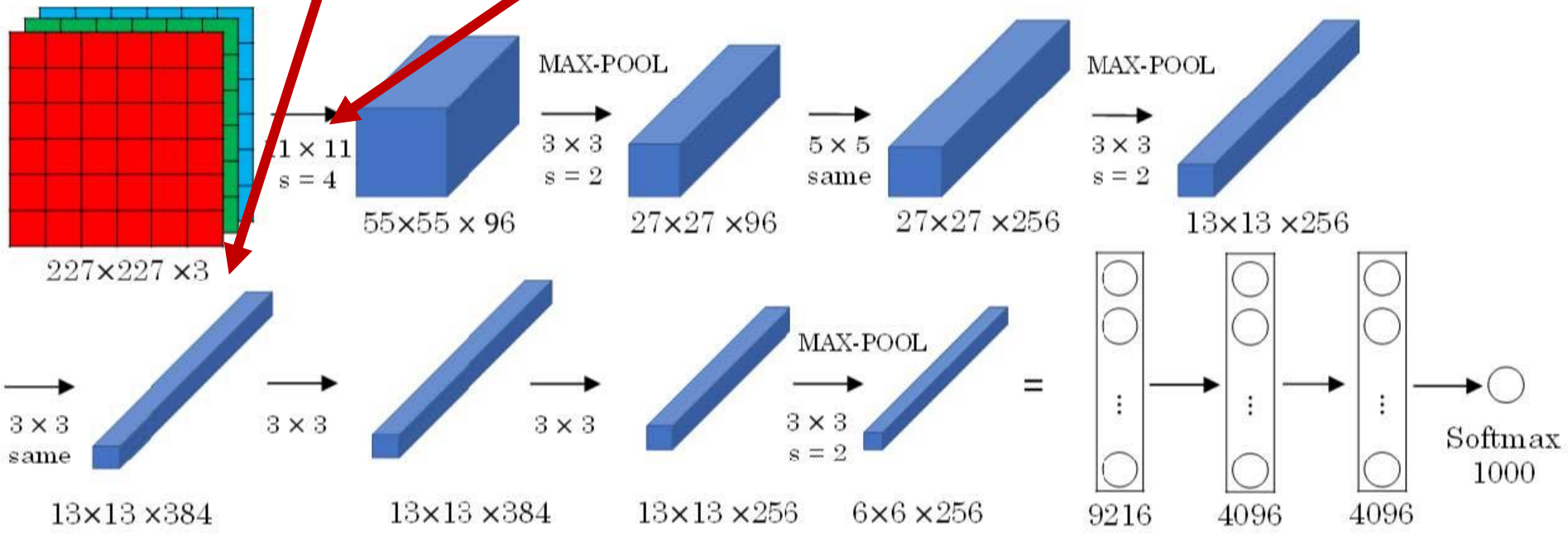


2012年, Hinton组Alex Krizhevsky参加ImageNet 竞赛, 使用 (Alexnet) 以超过第二名10个百分点夺冠，特点：首个被广泛关注的CNN 模型，首次采用ReLU激活函数，最大池化代替平均池化，添加Dropout层减小过拟合，采用数据扩增技术，使用2块GPU加速计算。

2012 Alexnet 结构

RGB3颜色分量
➔3个通道

卷积核= 11×11
步幅（stride)= 4

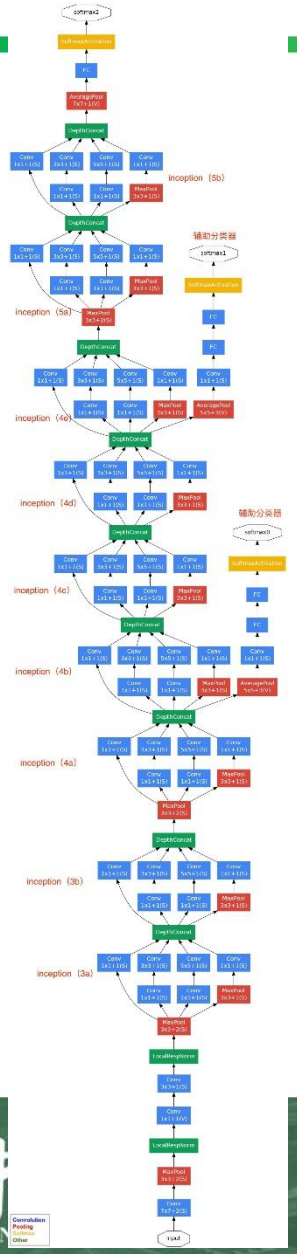


层与层之间的激活函数和Batch Normalization没有画出

GPU

- 中文：图形处理器（或显卡，显示核心、视觉处理器、显示芯片）
- 英文：GPU （Graphics Processing Unit）
- 定义：专门在个人电脑、工作站、游戏机和一些移动设备上做图像和图形相关运算工作的微处理器。
- 特点：
  - 早期的显卡只包含简单的存储器和帧缓冲区，只起了一个图形的存储和传递作用，一切操作都必须由CPU来控制。
  - 后来发展的显卡不单单存储图形，而且能完成大部分图形处理功能，这样就大大减轻了CPU的负担。
  - 线性计算的并行化的程度好，用来做Alexnet及ANN的后向传播的梯度更新。

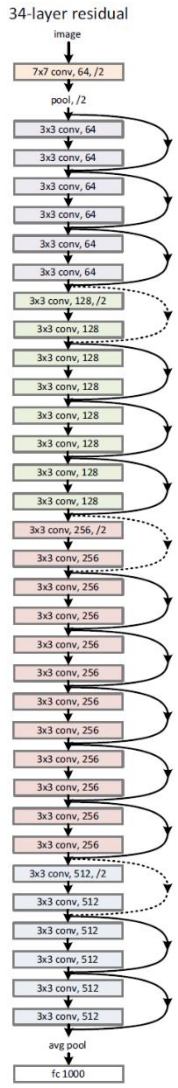
2014 GoogLenet

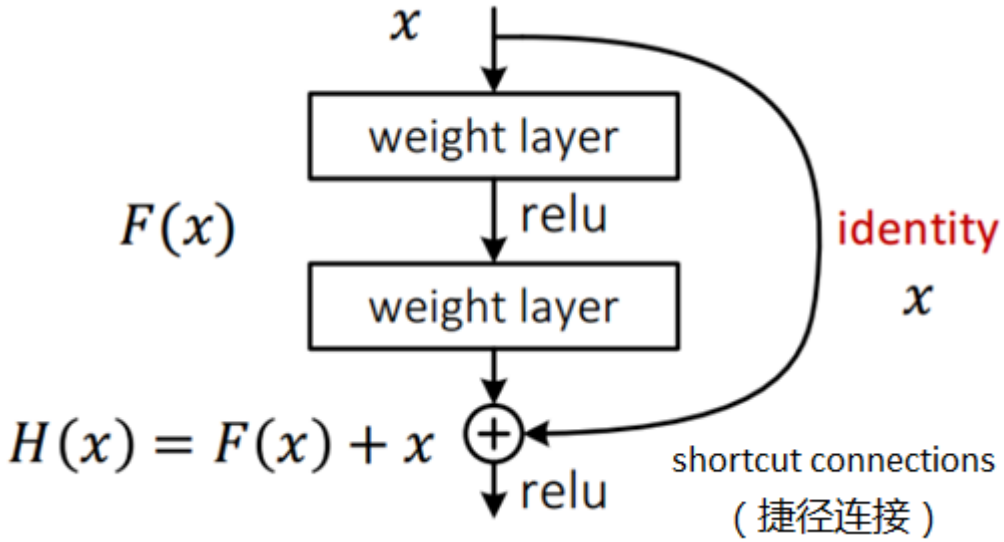


- 2014年，GoogLeNet（不是GoogleNet ）获得ImageNet挑战赛第一名
- 特点：
  - 采用模块化Inception结构，方便增添和修改；
  - 采用average pooling（平均池化）+全连接层；
  - 依然使用了Dropout；
  - 为了避免梯度消失，增加了2个softmax用于向前传导梯度（辅助分类器）。

https://my.oschina.net/u/876354/blog/1637819

2015年ResNet (深度残差网络)



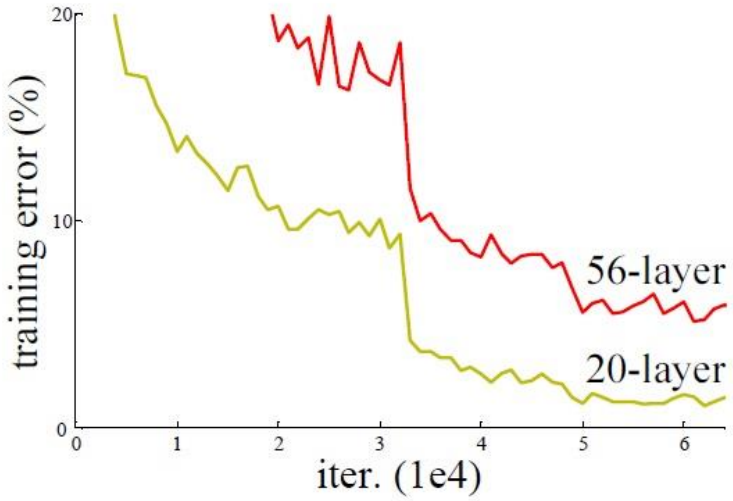




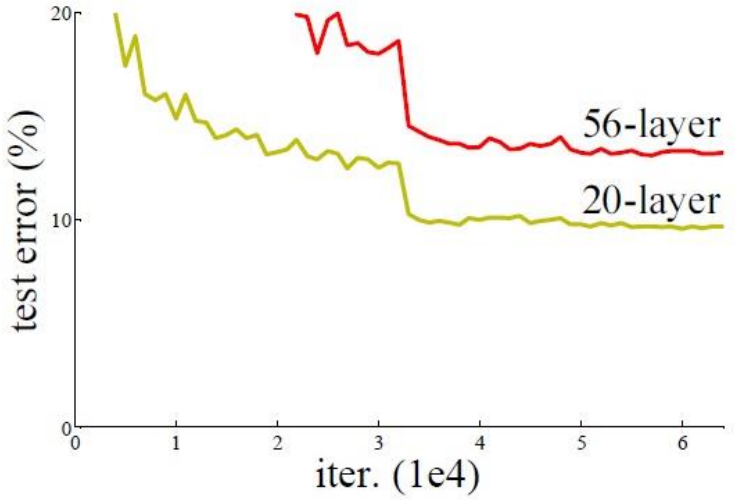
毕业于广州执信中学

何恺明与同事开发了ResNet，在ILSVRC2015竞赛中第1名，152层，将错误率降到了3.57。

ResNet 解决的问题



训练集误差图

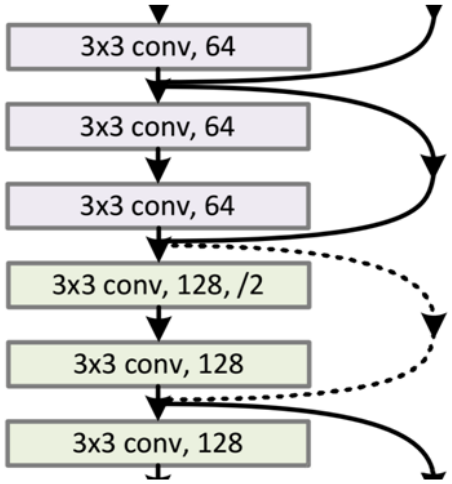


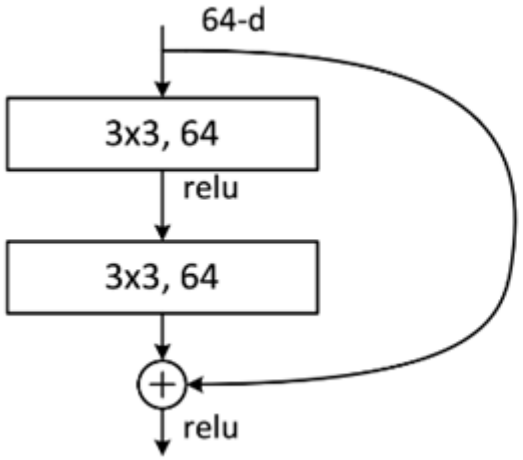
校验集误差图

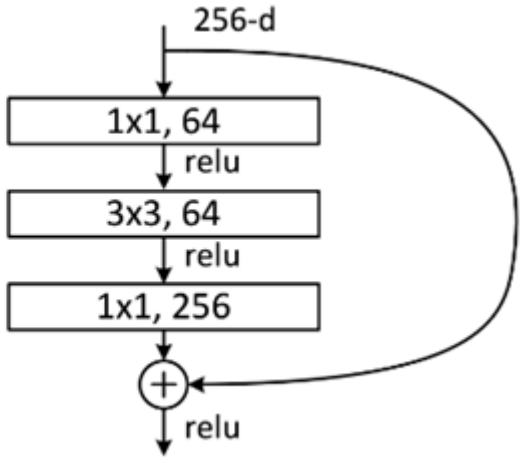
问题：普通网络变深以后，深度网络难以训练收敛➔梯度在传播过程中会逐渐消失（假如采用Sigmoid，信号为1，每向后传递一层，梯度就衰减为0.25，层数越多衰减越厉害），导致无法对前面层权重有效调整

目标：加深层数、避免梯度消失、提升模型精度?

ResNet (深度残差网络)







通过**捷径连接**的方式，将学习目标改变为：**F(X)= H(X) -x (残差)差值**，使到随着网络加深，准确率不下降，从而使得**层数达到上百层甚至千层**。

深度学习的概念

- **中文：深度学习**
- **英文：Deep Learning**
- **定义：** 深度学习是机器学习的一个分支，通过多层处理，逐渐将初始的“低层”特征表示转化为“高层”特征表示后，用“简单模型”即可完成复杂的分类、回归等问题。
- **特点：**
  - **强调了模型结构的深度**（有5、6，甚至10+以上的隐层）
  - **明确了特征学习的重要性**。即通过逐层特征变换，将样本在原空间的特征表示变换到一个新特征空间，从而使分类或预测更容易。与人工规则构造特征的方法相比，深度学习更能够刻画数据丰富的内在信息。
  - 在**语音和图像识别方面**取得的效果，**远超先前相关技术**。

深度学习的典型模型

输入	代表模型	典型应用	输出
序列 时序	RNN/LSTM	语音识别、翻译、自然语言处理、时序预测、文本生成等	数值 类别 序列
图像	CNN	人脸识别、运动检测和许多高级驾驶员辅助技术（如自动驾驶、车道检测、行人检测和自动泊车）等问题。	数值 类别 图像
数值 （序列）	BPANN	基于中小规模二维表格数据的回归和分类	数值 类别

循环神经网络（RNN）模型

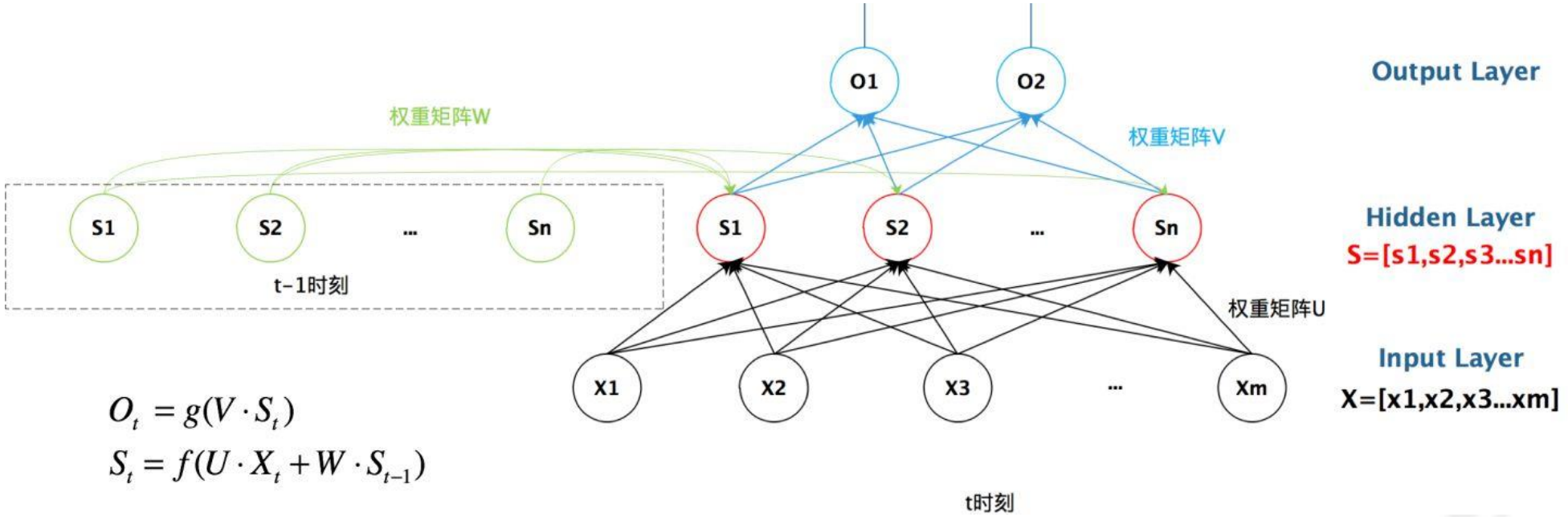
RNN模型的概念

- **中文：循环神经网络**
- **英文：Recurrent Neural Network（RNN)**
- **定义：是隐藏层的节点之间有连接，主要用于对序列数据进行分类、预测等处理的神经网络。**
- **功能：主要用于处理输入是序列或时序的问题**
- **主要公式：隐含层t时刻的状态，与t-1时刻的状态和t时刻的输入有关。**

$$h_t = f_W(h_{t-1}, x_t)$$

- h_t为t时刻的隐含层状态，
- x_t为t时刻的输入层输入。
- 请注意：f_W与t无关。

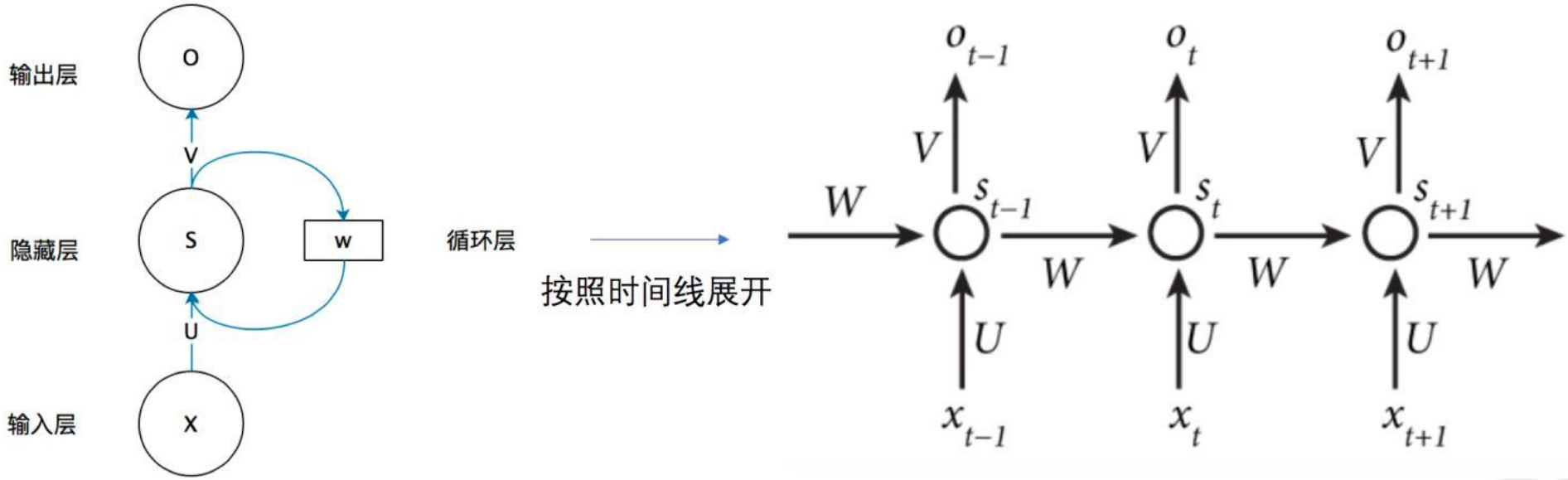
RNN模型的结构



$$O_t = g(V \cdot S_t)$$

$$S_t = f(U \cdot X_t + W \cdot S_{t-1})$$

RNN模型的结构


输出层
O
V
隐藏层
S
w
U
输入层
X
循环层
按照时间线展开
o_{t-1}
o_t
o_{t+1}
V
W
s_{t-1}
s_t
s_{t+1}
U
x_{t-1}
x_t
x_{t+1}

RNN的计算公式

简写公式：

$$O_t = g(V \cdot S_t)$$

$$S_t = f(U \cdot X_t + W \cdot S_{t-1})$$

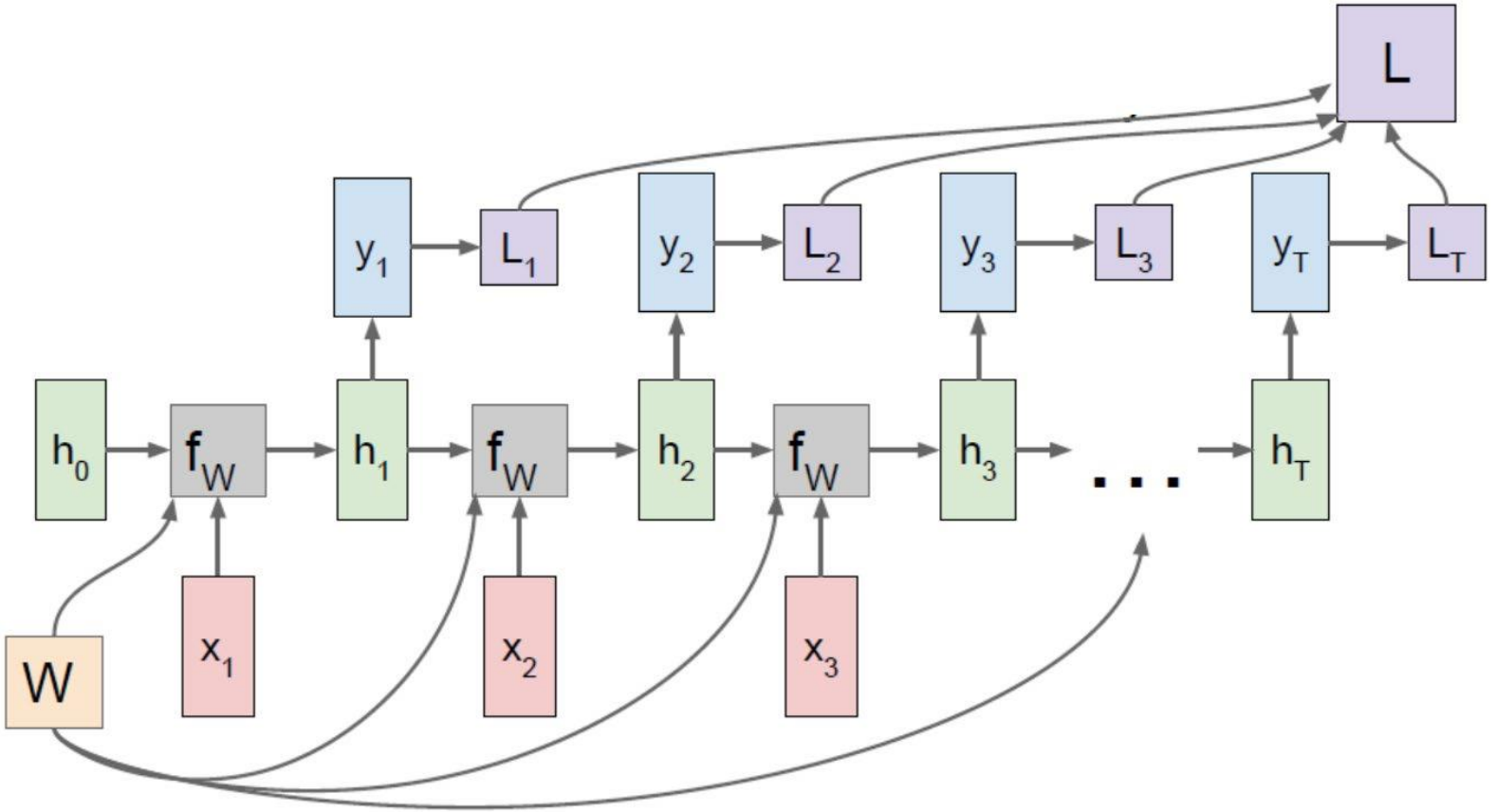
St的值不仅仅取决于Xt，还取决于St-1

展开公式：

$$\begin{aligned}
o_t &= g(Vs_t) && (3)\\
&= Vf(Ux_t + Ws_{t-1}) && (4)\\
&= Vf(Ux_t + Wf(Ux_{t-1} + Ws_{t-2})) && (5)\\
&= Vf(Ux_t + Wf(Ux_{t-1} + Wf(Ux_{t-2} + Ws_{t-3}))) && (6)\\
&= Vf(Ux_t + Wf(Ux_{t-1} + Wf(Ux_{t-2} + Wf(Ux_{t-3} + \dots)))) && (7)
\end{aligned}$$

为什么RNN可以往前看任意多个输入值的原因。

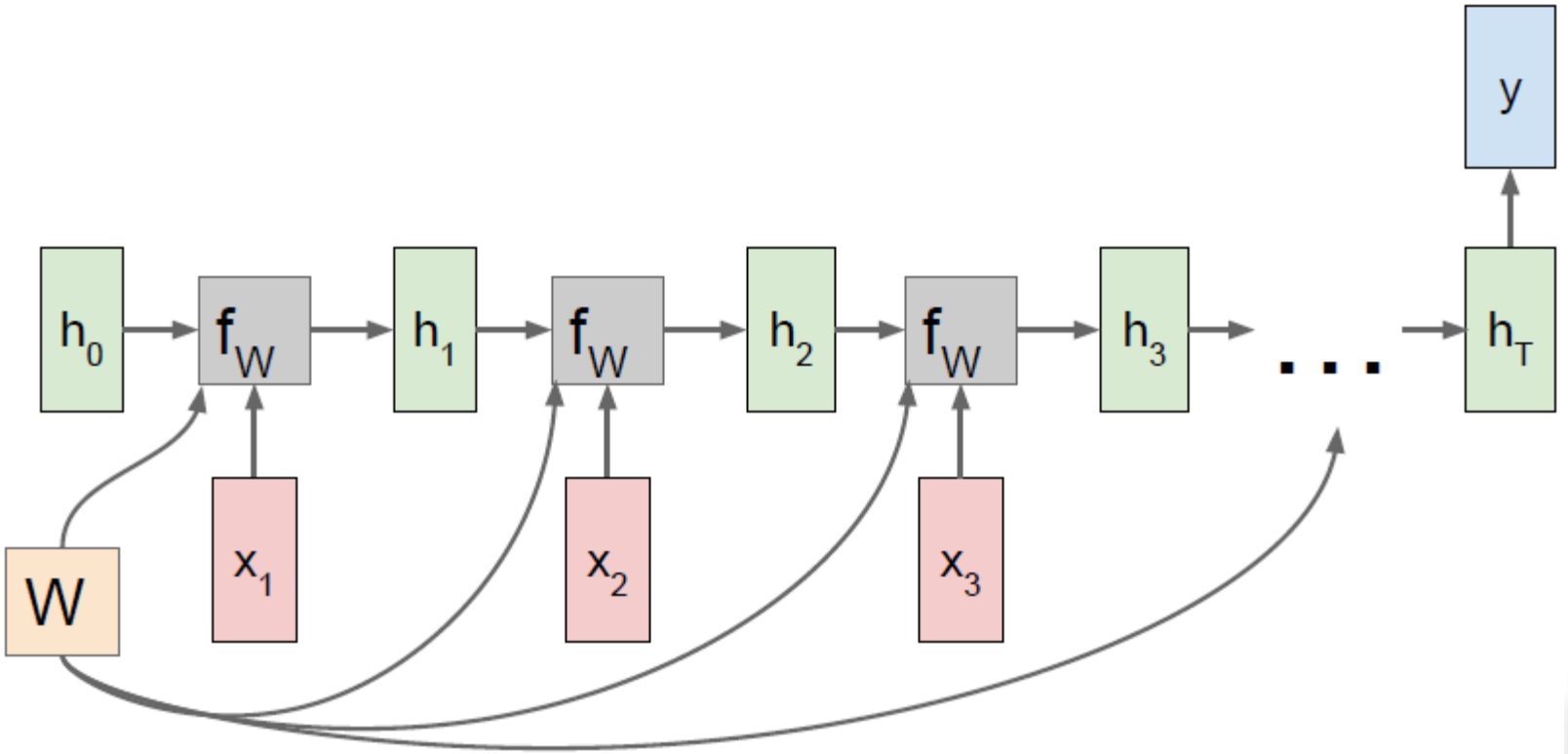
RNN结构1：输入与输出多对多



典型应用：大词汇连续的语音识别、机器翻译

中山大學 SUN YAT-SEN UNIVERSITY

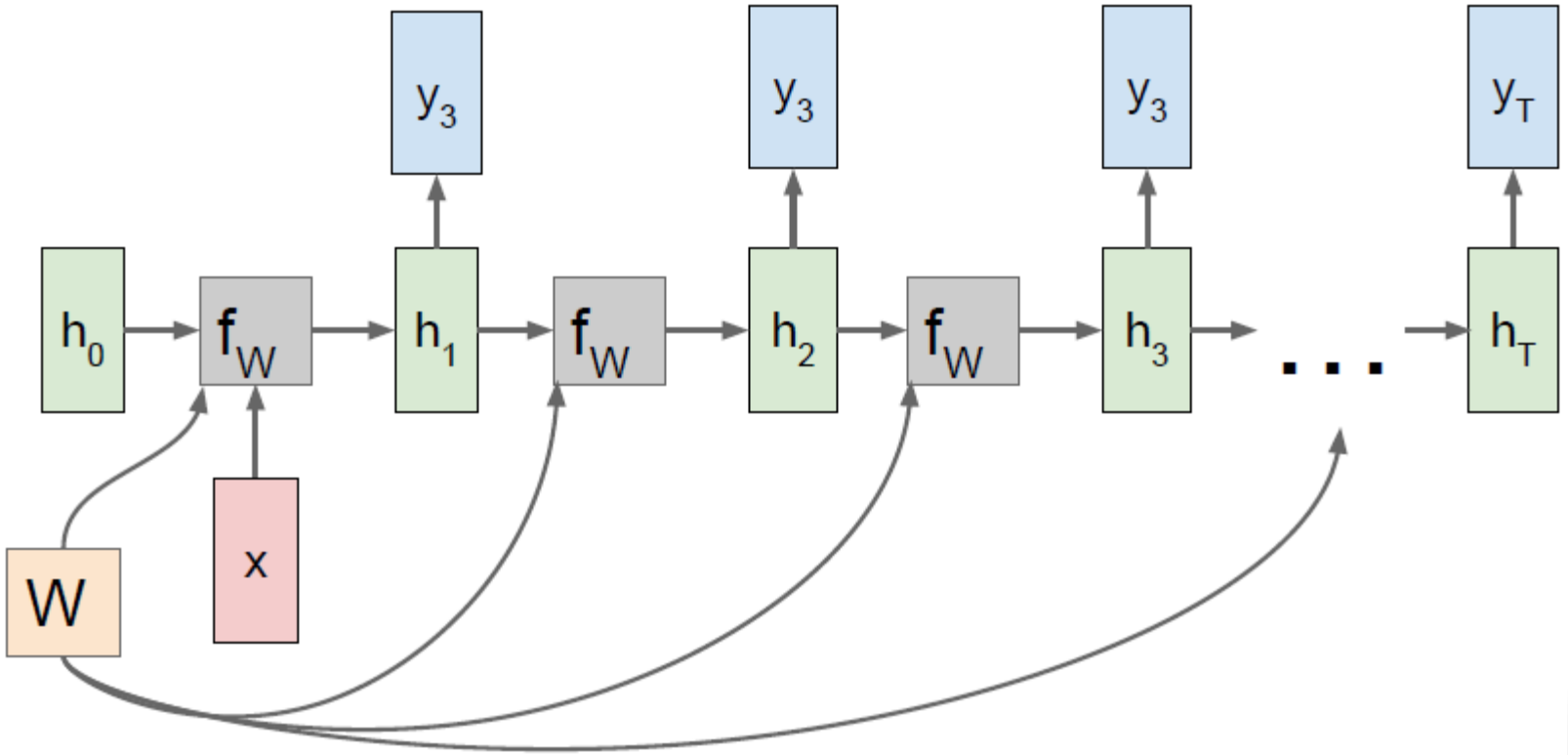
RNN结构2：输入与输出多对一



典型应用：动作识别、行为识别、单词量有限的语音识别

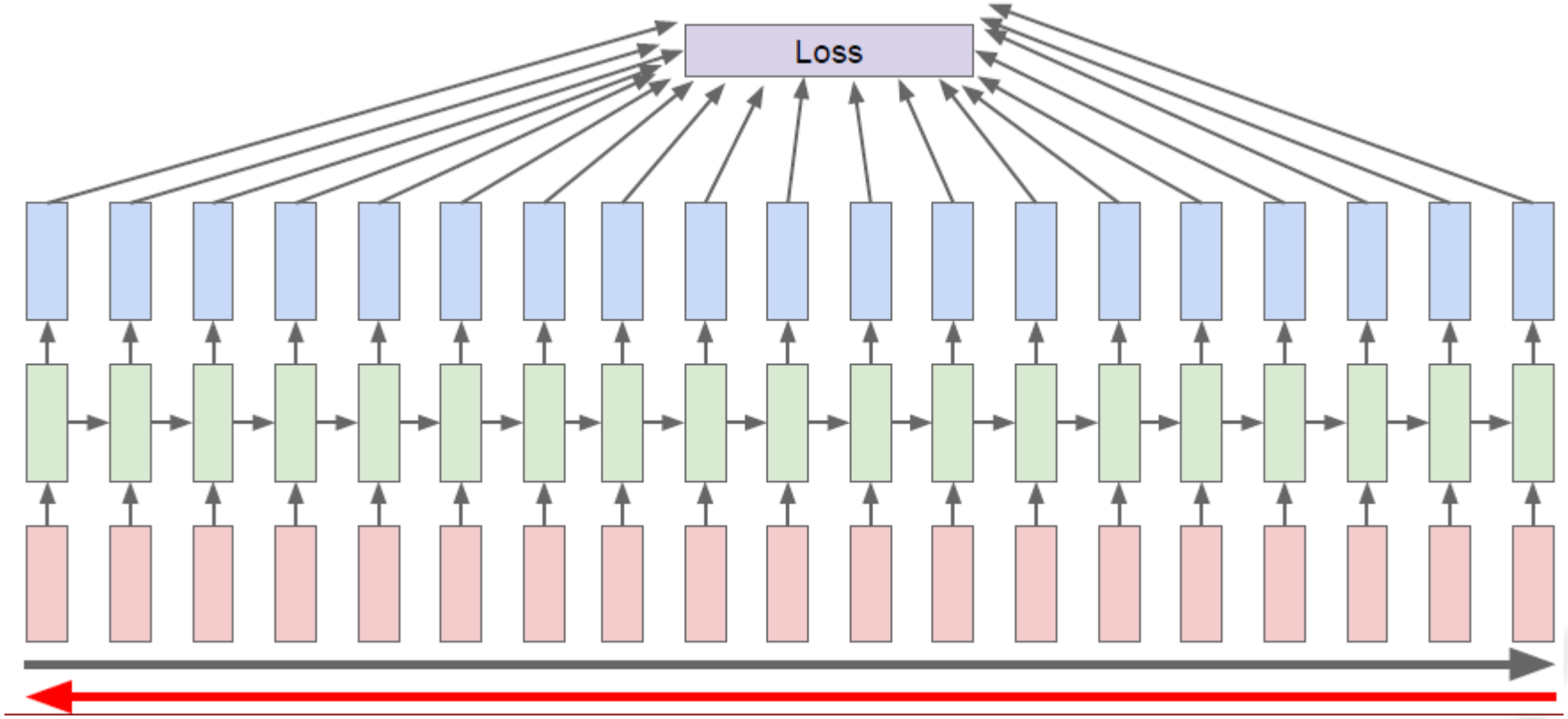
中山大
SUN YAT-SEN UNIV

RNN结构3：输入与输出一对多



典型应用：文本生成、图像文字标注

RNN的训练方式1

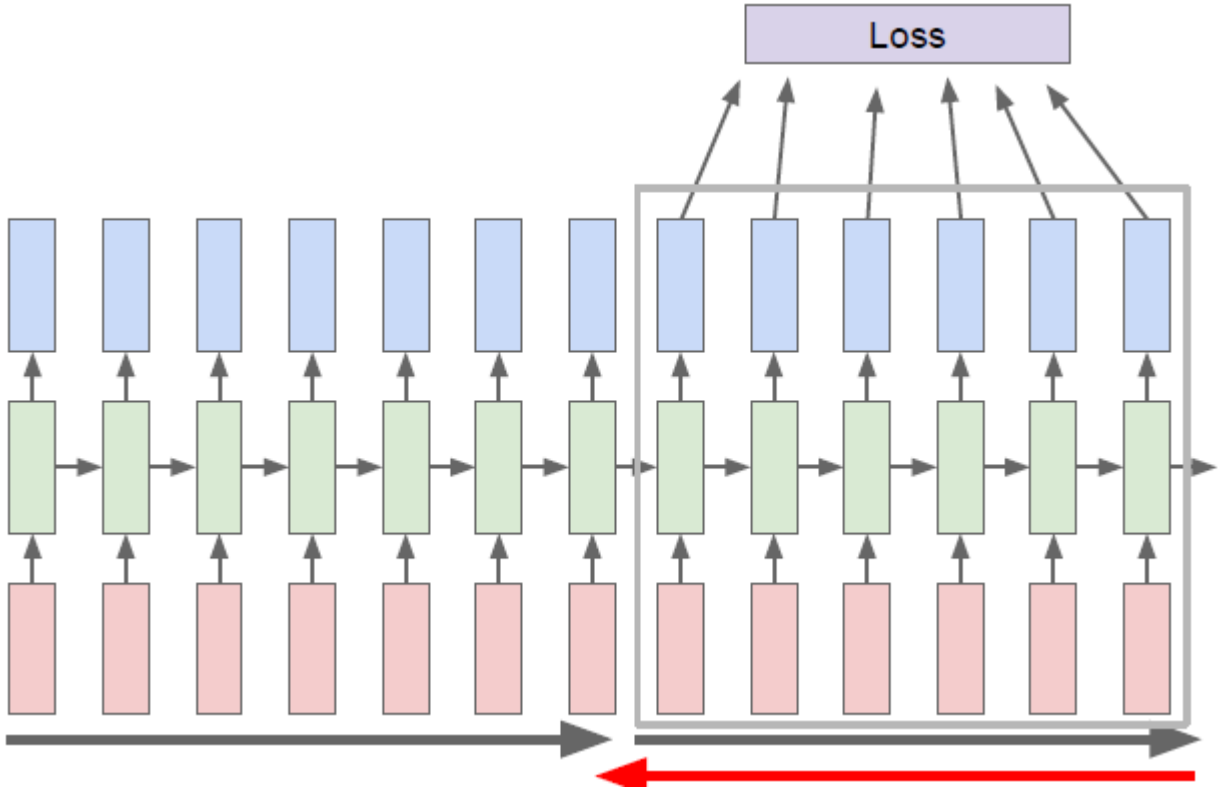


结构：将RNN沿时间轴展开。

训练：前向计算后，将每一个时刻t的LOSS相加作为总的目标函数，逐级求导。

中山大
SUN YAT-SEN UNI

RNN的训练方式2



结构：将RNN沿时间轴展开。

训练：前向计算后，将RNN截断有限步，实际中只求有限步的导数。

中山大
SUN YAT-SEN UNI

RNN的典型应用1：文本生成

训练样本：
中国古诗集

首春:寒随穷律变，春逐鸟声开。初风飘带柳，晚雪间花梅。碧林青旧竹，绿沼翠新苔。芝田初雁去，绮树巧莺来。

初晴落景:晚霞聊自怡，初晴弥可喜。日晃百花色，风动千林翠。池鱼跃不同，园鸟声还异。寄言博通者，知予物外志。

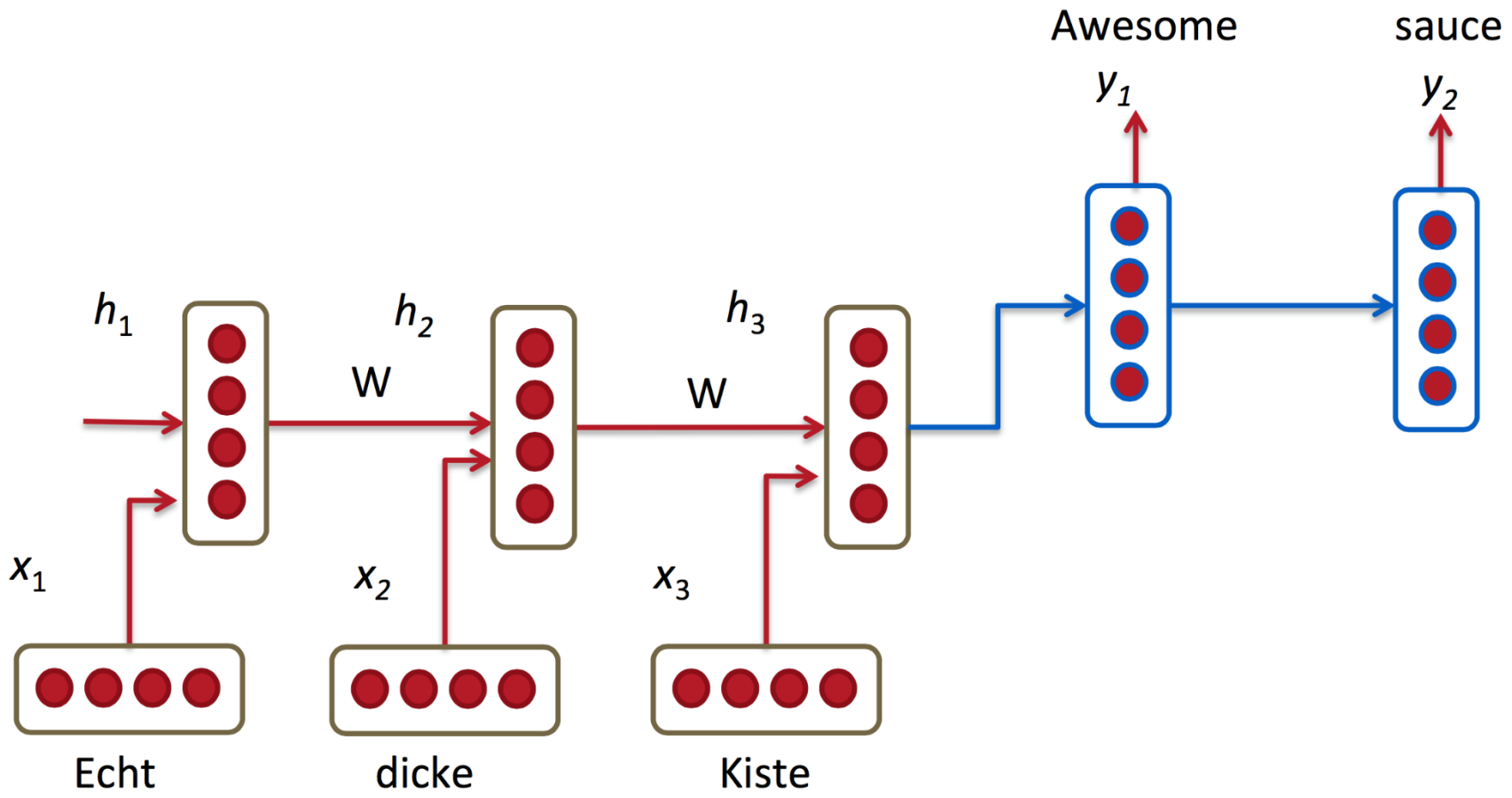
初夏:一朝春夏改，隔夜鸟花迁。阴阳深浅叶，晓夕重轻烟。哢莺犹响殿，横丝正网天。珮高兰影接，绶细草纹连。碧鳞惊棹侧，玄燕舞檐前。何必汾阳处，始复有山泉。

度秋:夏律昨留灰，秋箭今移晷。峨嵋岫初出，洞庭波渐起。桂白发幽岩，菊黄开灞涘。运流方可叹，含毫属微理。

预测：
新诗集

1. 门前车马喧，时闻犬吠唱。向来谢公相，此生自引领。
2. 生涯已做平原客，不及当年有古人。关塞门前谁可见，白云亭上倍凄凉。
3. 风霜初一夜，地暖不生烟。印出泥中火，晴天月白明。
4. 谁能牧儿女，邯郸亦无求。新诗初入讽，往事愧不磨。

RNN的典型应用2：翻译



RNN的优点

- 处理任意长度的输入
- 模型形状不随输入长度增加
- 计算考虑了历史信息
- 权重随时间共享

RNN的缺点

- 计算速度慢
- 难以获取很久以前的信息➔很难捕捉到长期的依赖关系，
- 因为乘法梯度可以随着层的数量呈指数递减/递增，在RNN中经常遇到梯度消失和爆炸现象。➔在某些类型的RNN中使用特定的门结构
- 无法考虑当前状态的任何未来输入

长短期记忆网络（LSTM）模型

LSTM的概念

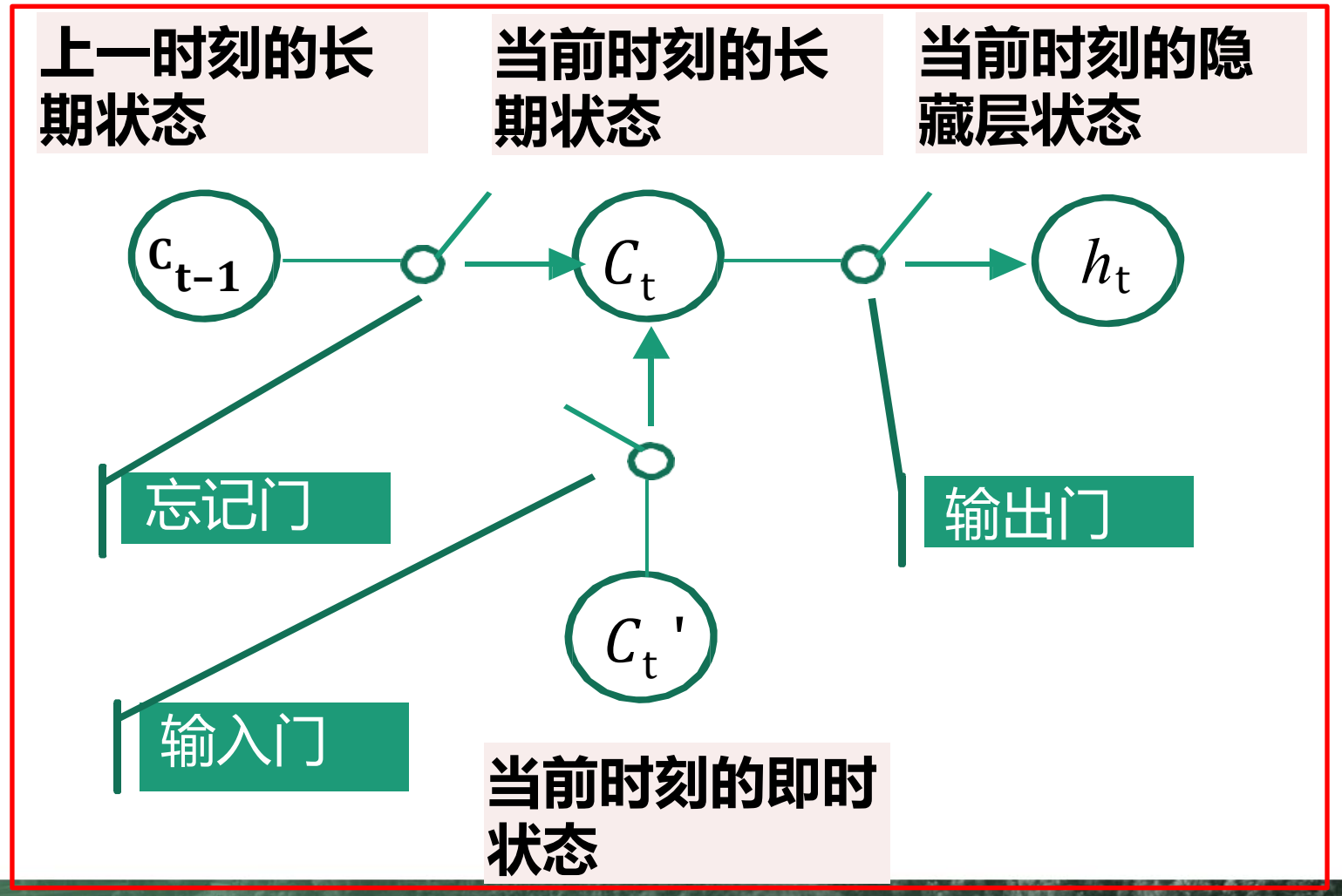
- **中文：长短期记忆网络**
- **英文：Long Short Term Memory Network (LSTM)**
- **功能：主要用于处理输入是序列或时序的问题**
- **特点：**
  - **是RNN中的一种，增加了RNN中单元的复杂度，使模型更复杂，增加系统表现力。**
  - **LSTM的误差反向传播更方便和直接，梯度更新避免了RNN长期依赖导致的的爆炸或消失现象。**
  - **建议涉及RNN的应用都用LSTM或LSTM相关的变种。**

LSTM的结构特点

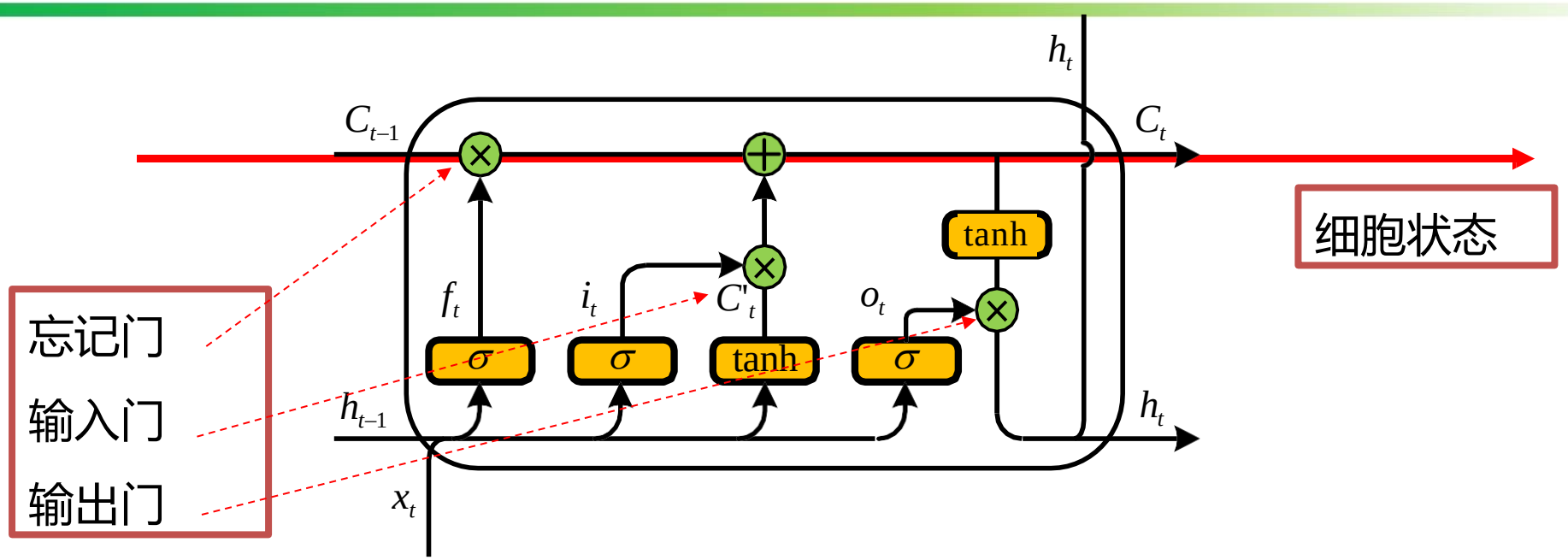
- 相比RNN， LSTM引入了细胞状态概念。
  - RNN只考虑最近的状态，
  - LSTM的细胞状态会决定哪些状态应该被留下来，哪些状态应该被遗忘

- LSTM分成了四个部分：
  1）哪些细胞长期状态应该被遗忘（遗忘门控制）
  2）哪些当前即时状态应该被输入（输入门控制）
  3）当前长期状态=过去长期状态+当前即时状态（不控制）
  4）哪些当前长期状态应该被输出给隐含层（输出门控制）

LSTM结构


上一时刻的长期状态
当前时刻的长期状态
当前时刻的隐藏层状态
c_{t-1}
C_t
h_t
忘记门
输出门
C_t'
输入门
当前时刻的即时状态

LSTM结构



⇒ 神经网络层

⇒ 逐点操作

⇒ 传输向量

⇒ 向量被复制

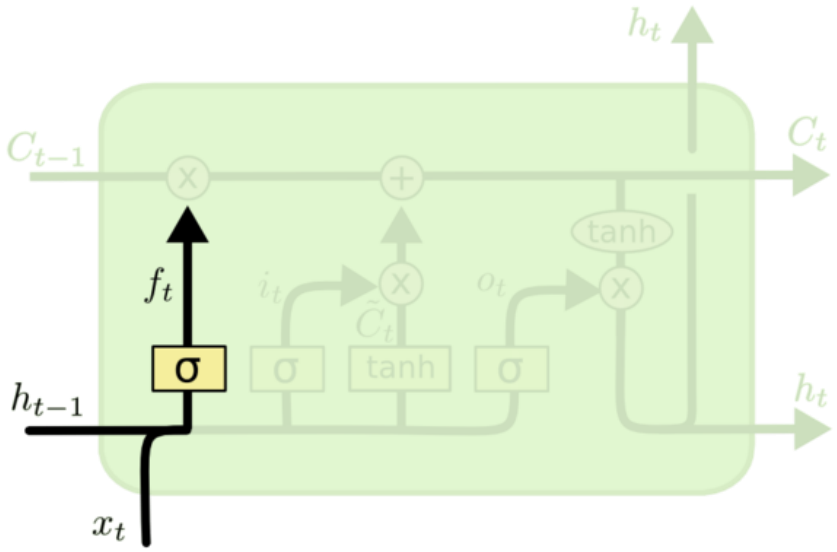
⇒ 向量的连接

乘法门：信息选择通过

加法门：信息合并通过

LSTM结构分解

哪些细胞长期状态是否被遗忘（遗忘门控制）



遗忘门

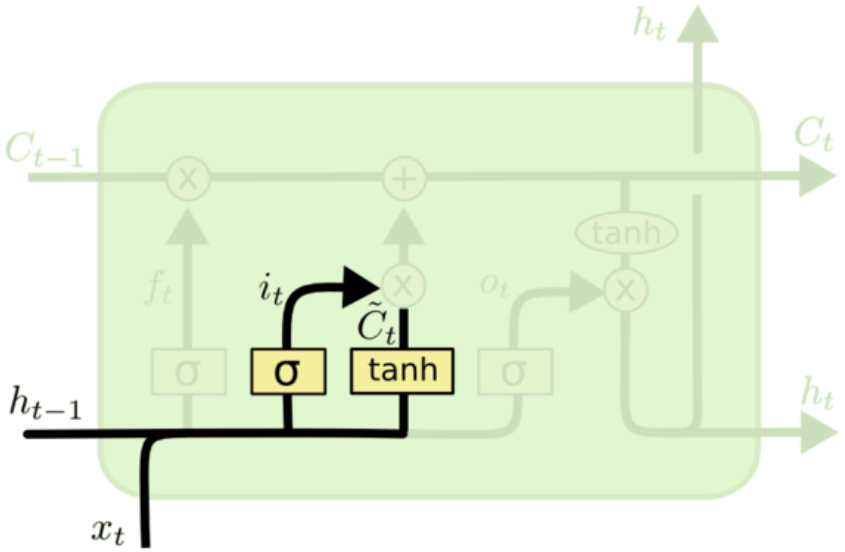
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

隐含层过去状态

输入层当前状态

LSTM结构分解

哪些当前即时状态是否被输入（输入门控制）



输入门

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

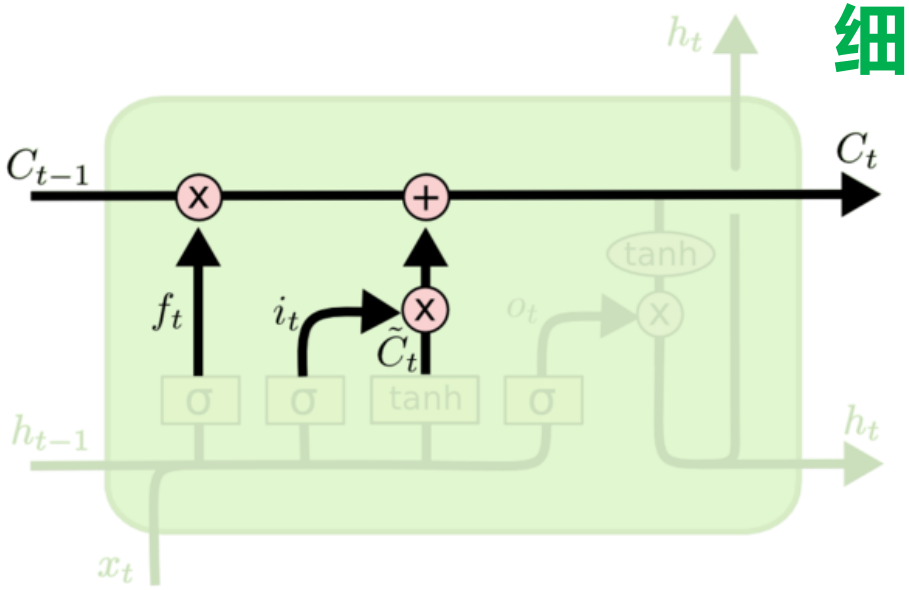
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

当前细胞即时状态

隐含层过去状态　输入层当前状态

LSTM结构分解

当前长期状态=过去长期状态+当前即时状态（不控制）



细胞当前长期状态

遗忘门　　**输入门**

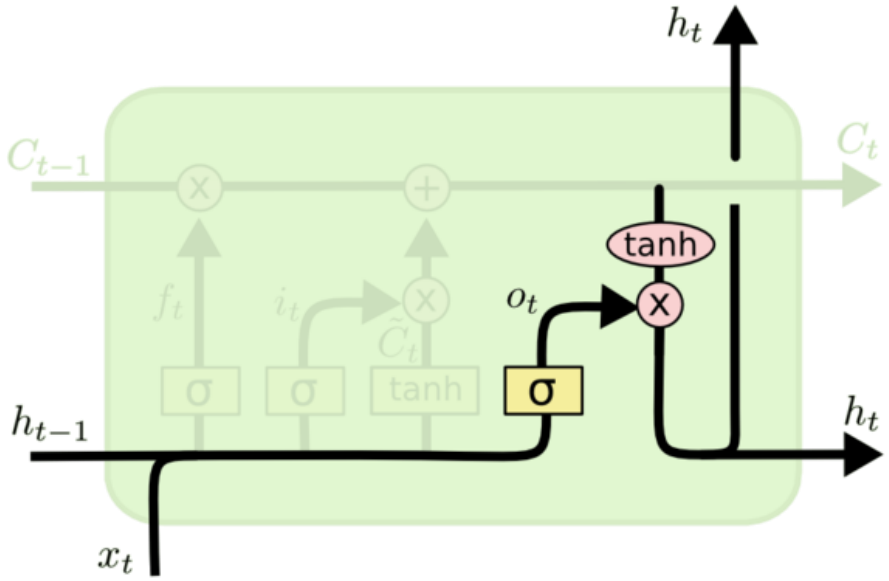
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

细胞过去长期状态

当前细胞即时状态

LSTM结构分解

哪些当前长期状态是否被输出给隐含层（输出门控制）



输出门

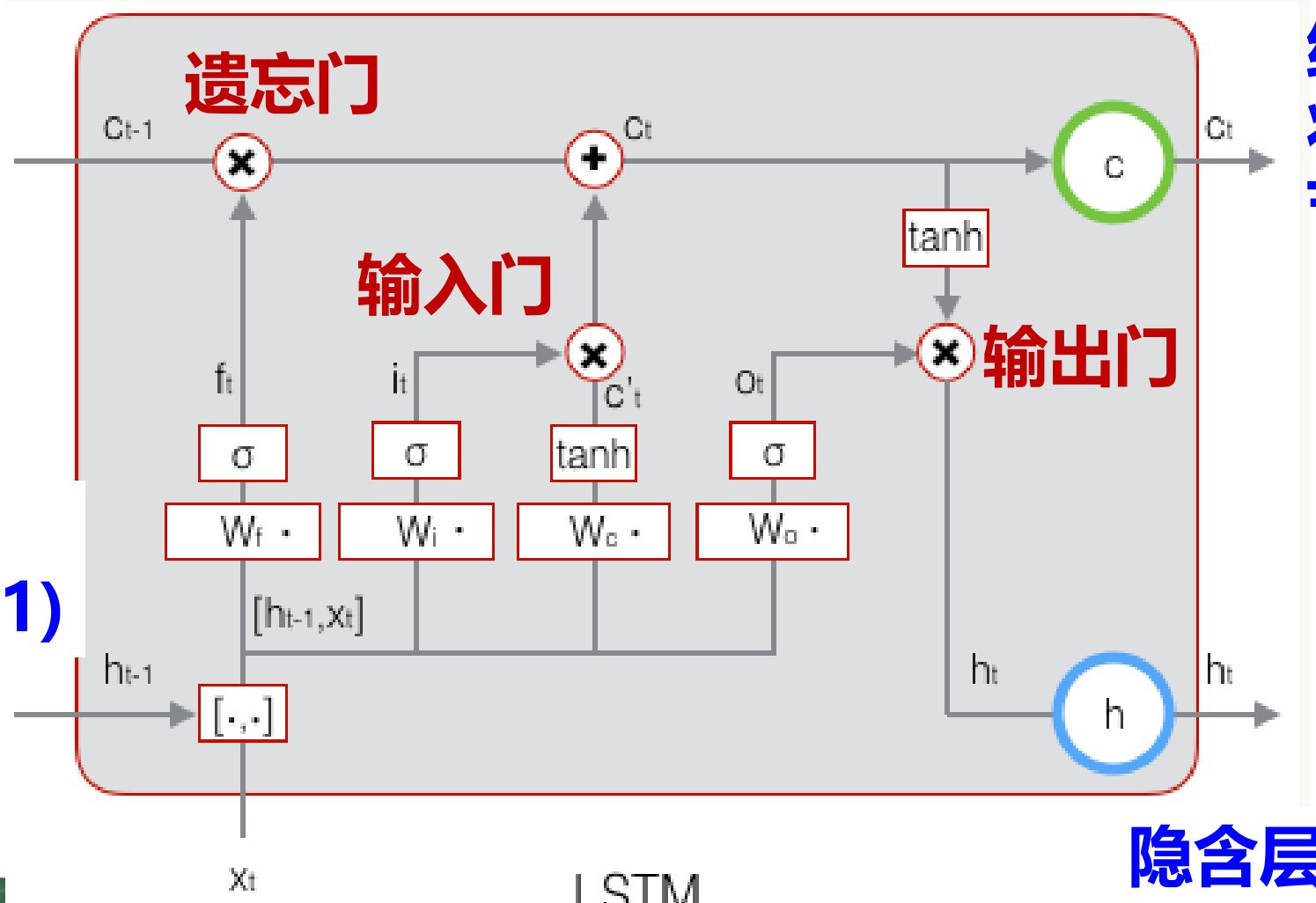
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

当前隐含层状态

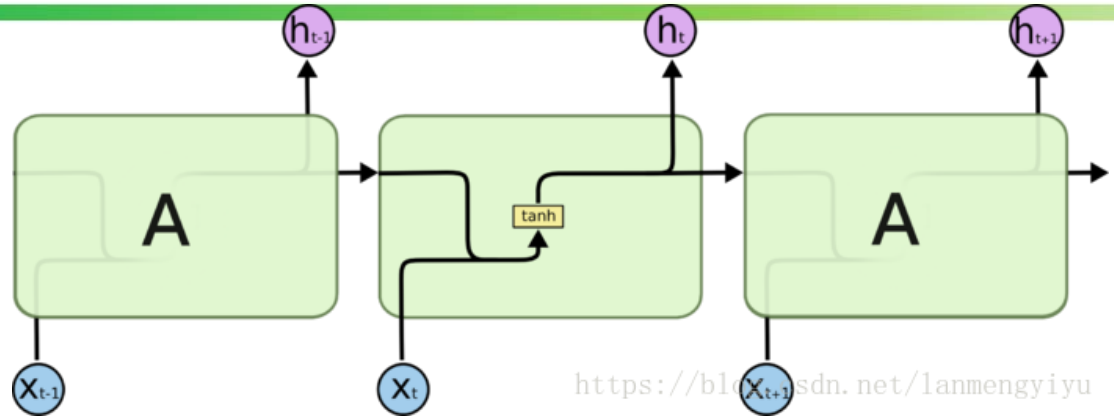
当前细胞长期状态

LSTM结构分解


遗忘门
输入门
输出门
细胞状态节点
隐含层节点(t-1)
隐含层节点(t)
输入层节点
LSTM
Ct-1
Ct
ht-1
ht
xt
ft
it
C't
Ot
σ
tanh
Wf ·
Wi ·
Wc ·
Wo ·
[ht-1,xt]
[·,·]
c
h

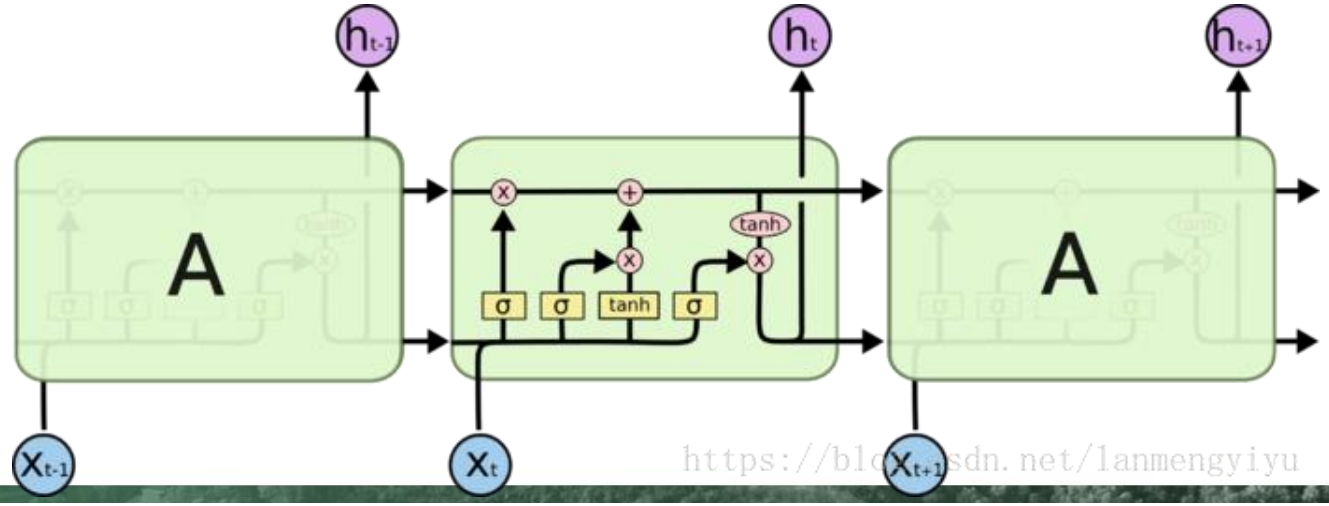
LSTM和RNN的比较

RNN



RNN中，将过去输出和当前输入连环到一起，通过tanh来控制两者输出，只考虑最近时刻的状态。RNN有两个输入和一个输入。

LSTM



LSTM为了能记住长期的状态，在RNN的基础上增加了一路输入和一路输出，增加的就是细胞状态

LSTM的结构特点

特征学习
- –忘记门
- –输入门
- –输出门

增加深度
- –隐含层（t, t-1, … t-i）
- –细胞状态（t, t-1 , … t-i ）

深度学习的典型模型

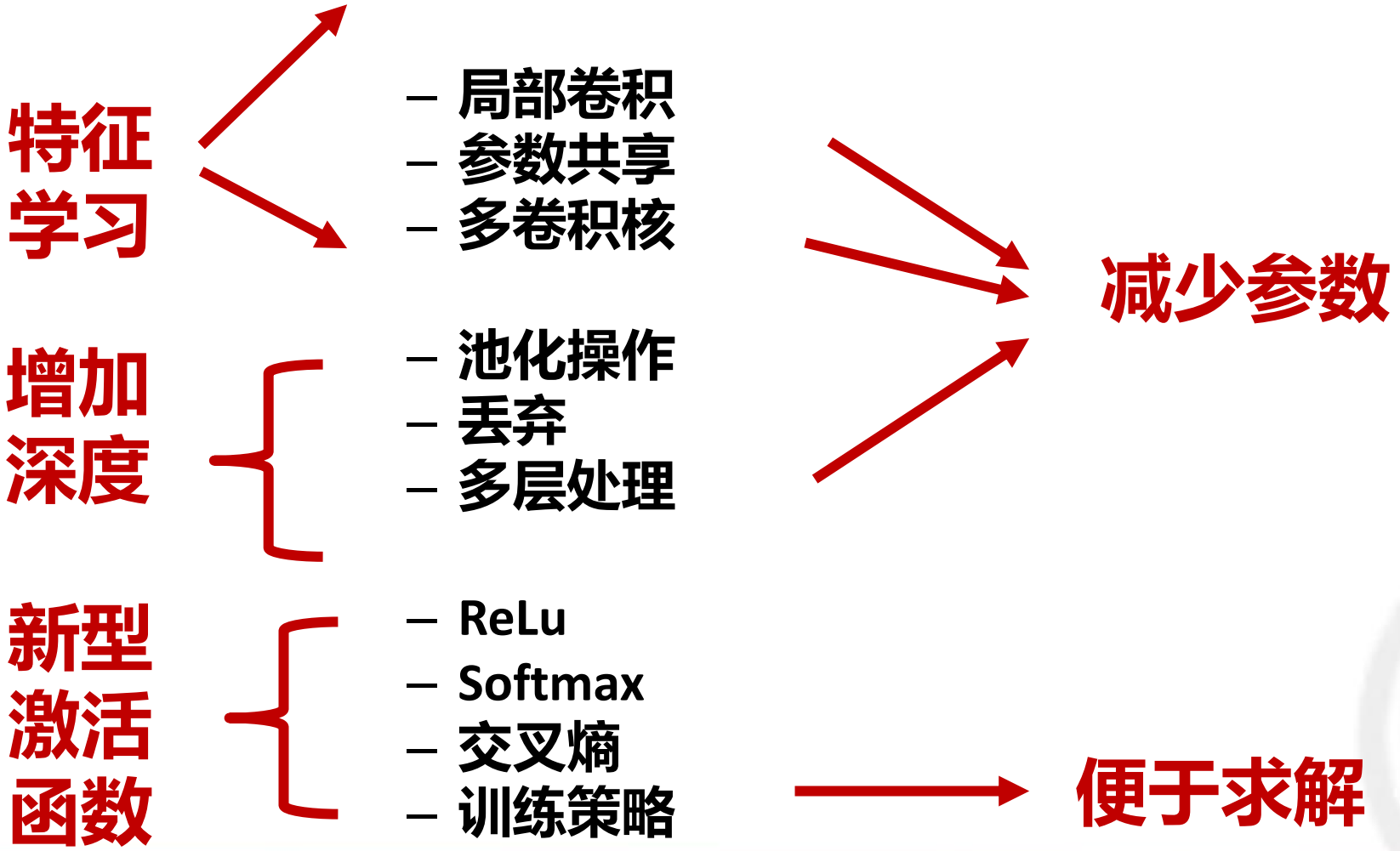
输入	代表模型	典型应用	输出
序列 时序	RNN/LSTM	语音识别、翻译、自然语言处理、时序预测、文本生成等	数值 类别 序列
图像	CNN	人脸识别、运动检测和许多高级驾驶员辅助技术（如自动驾驶、车道检测、行人检测和自动泊车）等问题。	数值 类别 图像
数值 （序列）	BPANN	基于中小规模二维表格数据的回归和分类	数值 类别

卷积神经网络（CNN）模型

卷积神经网络（CNN）的概念

- **中文：** 卷积神经网络
- **英文：** Convolutional Neural Networks（CNN）
- **功能：** 主要用于处理输入是图像数据的问题
- **定义：** 是一类包含卷积计算且具有深度结构的前馈神经网络，是深度学习的代表算法之一 。
- **由来：** CNN仿造生物的视知觉（visual perception）机制构建，能够进行平移不变分类。
- **历史：** 对CNN的研究始于二十世纪80至90年代， LeNet-5是较早出现的卷积神经网络。

CNN的结构特点


特征学习
– 局部卷积
– 参数共享
– 多卷积核
增加深度
– 池化操作
– 丢弃
– 多层处理
减少参数
新型激活函数
– ReLu
– Softmax
– 交叉熵
– 训练策略
便于求解

卷积具体实现举例



原始图像

$$\begin{vmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{vmatrix}$$



特征图（feature map）

0	0	0	0	0	0	0	0	0	0
0	2	2	2	2	9	9	9	9	0
0	2	2	2	2	9	9	9	9	0
0	2	2	2	2	9	9	9	9	0
0	2	2	2	2	9	9	9	9	0
0	9	9	9	9	9	9	9	9	0
0	9	9	9	9	9	9	9	9	0
0	9	9	9	9	9	9	9	9	0
0	9	9	9	9	9	9	9	9	0
0	0	0	0	0	0	0	0	0	0

局部窗口

2	2	2
2	2	2
2	2	2

*

-1	-1	-1
-1	8	-1
-1	-1	-1

卷积核

2*(-1)	2*(-1)	2*(-1)
2*(-1)	2*(8)	2*(-1)
2*(-1)	2*(-1)	2*(-1)

矩阵元素求和

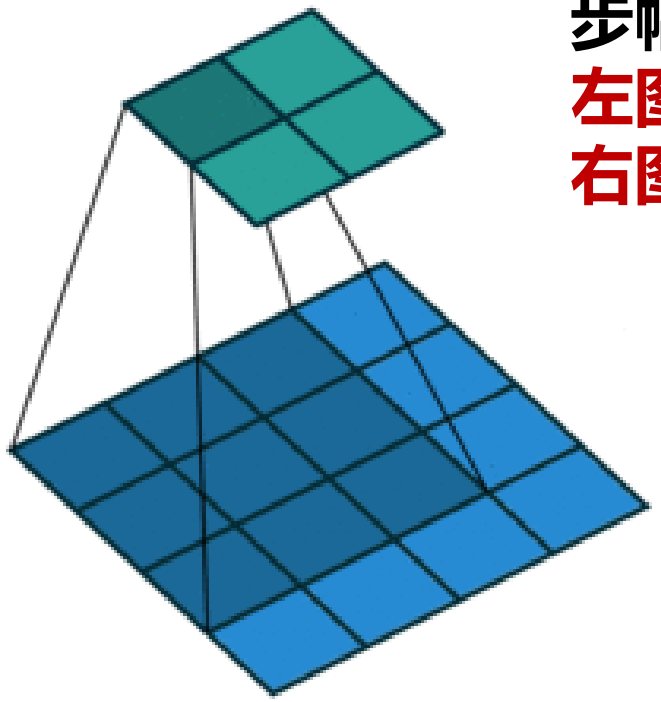
10	6	6	-8	41	27	27	45
6	0	0	-21	21	0	0	27
6	0	0	-21	21	0	0	27
-8	-21	-21	-35	14	0	0	27
41	21	21	14	7	0	0	27
27	0	0	0	0	0	0	27
27	0	0	0	0	0	0	27
45	27	27	27	27	27	27	45

哈达马积（hadamard product）

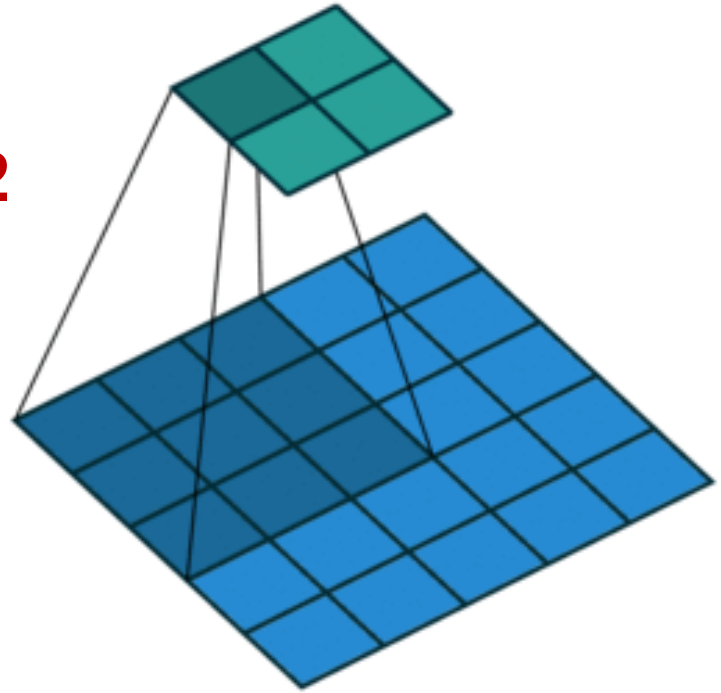
卷积（输入，滤波器）

输出

输入



步幅：
左图为1
右图为 2

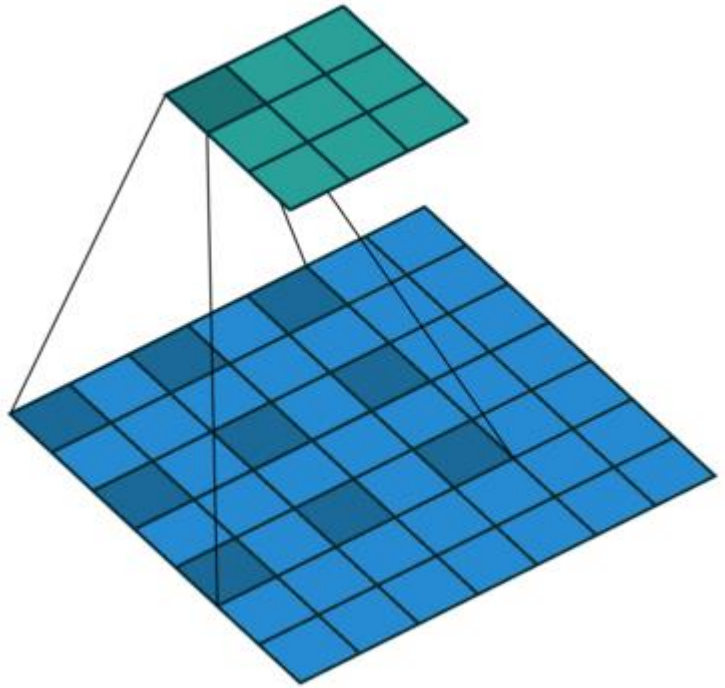


应用于图像中某个区域的一组权重称为**滤波器**。滤波器对输入进行**卷积**。扫描输入的 3×3 滤波器，滤波器移动的步长大小称为**步幅**。滤波器中的权重数为 h*w*c， h 是高度，w 是宽度，c 是输入中的通道数。

中山大
SUN YAT-SEN UNI

扩张卷积

输出



输入

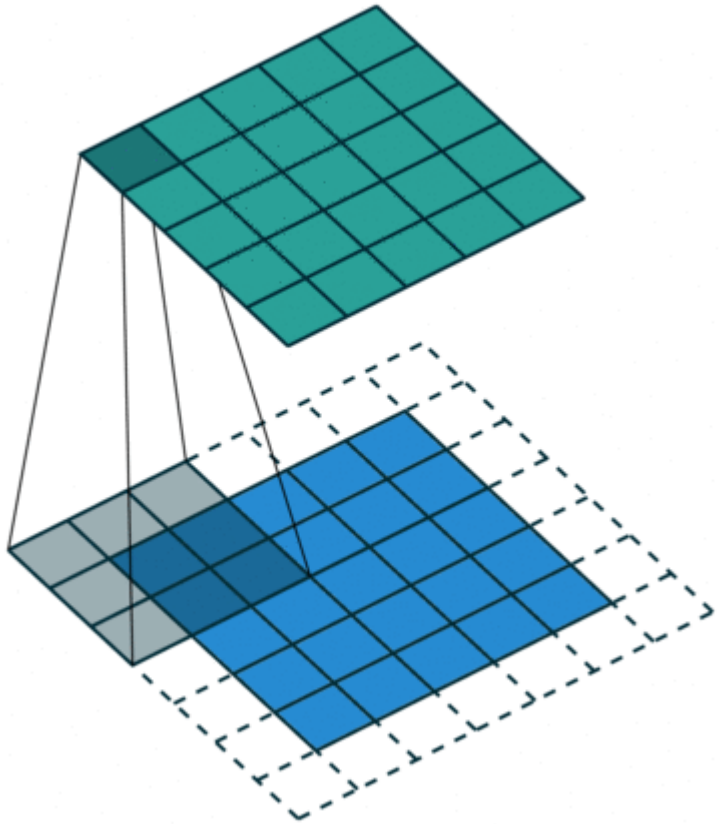
扩张卷积：通过在滤波器元素之间插入空白来扩展滤波器。**特点**：可增加层的**感受野（层可以看到的输入区域）**，而不增加参数个数或计算量。下图显示一个使用扩张系数 2 扫描输入来扩张的 3×3 滤波器，等效于在元素间插入零的 5×5 滤波器。

中山大
SUN YAT-SEN UN

填充+卷积

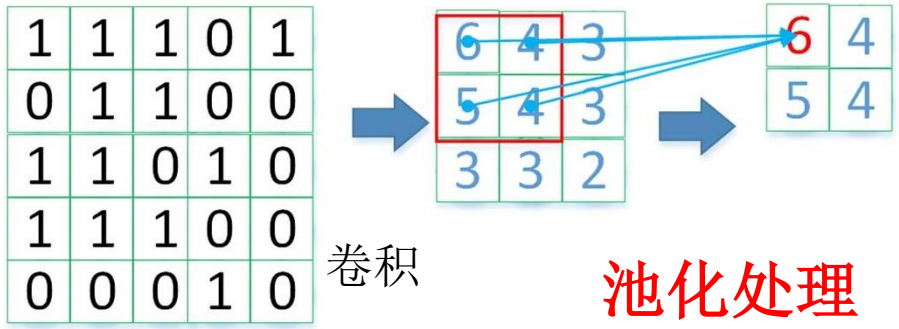
输出

输入



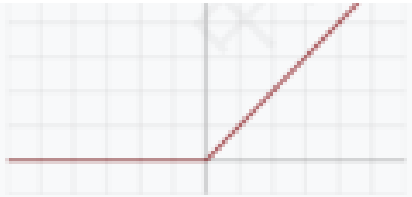
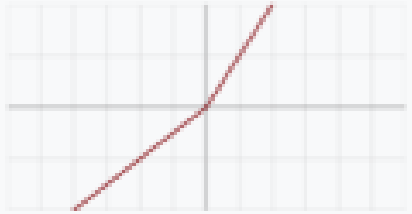
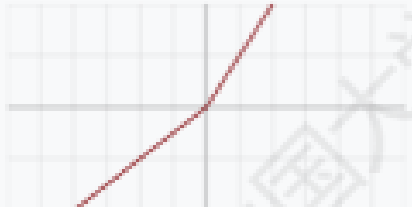
填充是指将向输入图像的边界追加值以增大其大小。通过调整零填充，您可以控制层的输出大小。此图是用于扫描输入的 3×3 滤波器，其零填充大小为 1。

池化处理实现举例



- **池化处理**也叫作降采样处（down-pooling），是对不同位置的特征 进行聚合统计。
- **算法**：取对应位置的最大值（最大池化）、平均值（平均池化）等。
- **优点**：1）降维；2）克服过拟合；3）在图像识别领域，池化还能提供平移和旋转不变性。

新型的激活函数（层）

Rectified linear unit (ReLU)[10]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$
Leaky rectified linear unit (Leaky ReLU)[11]		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$
Parameteric rectified linear unit (PReLU)[12]		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$

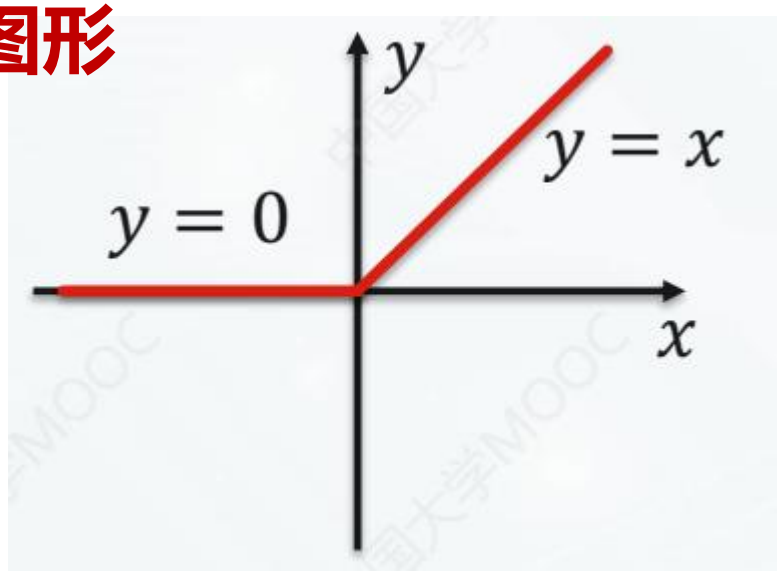
- **Relu为代表的新型的激活函数（层）**

ReLu层（激活函数）

形式

$$f(x)=\begin{cases}0, x<0\\ x, x\geq 0\end{cases}$$

图形



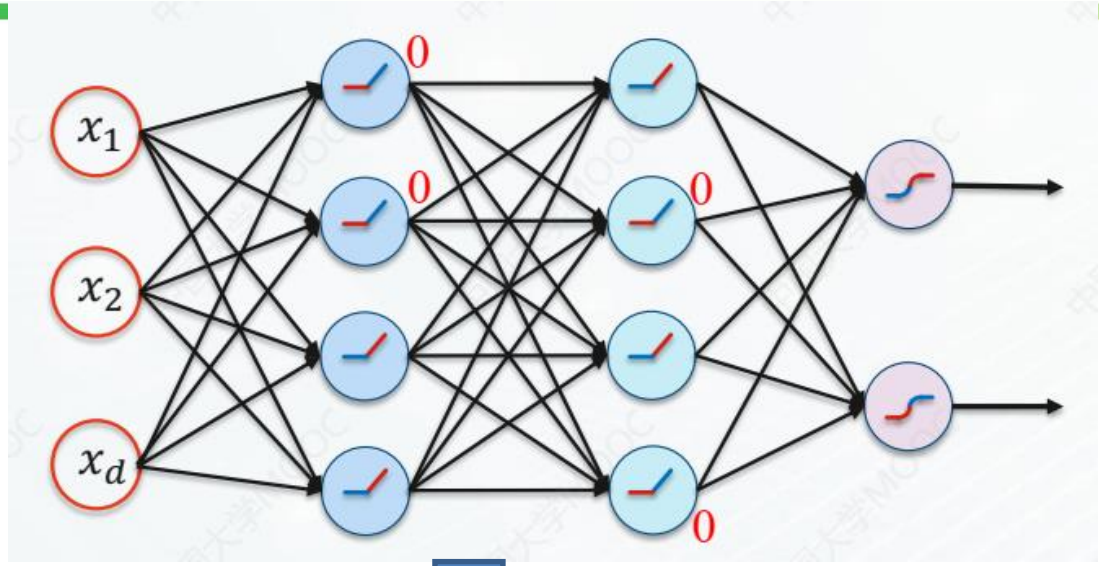
导数

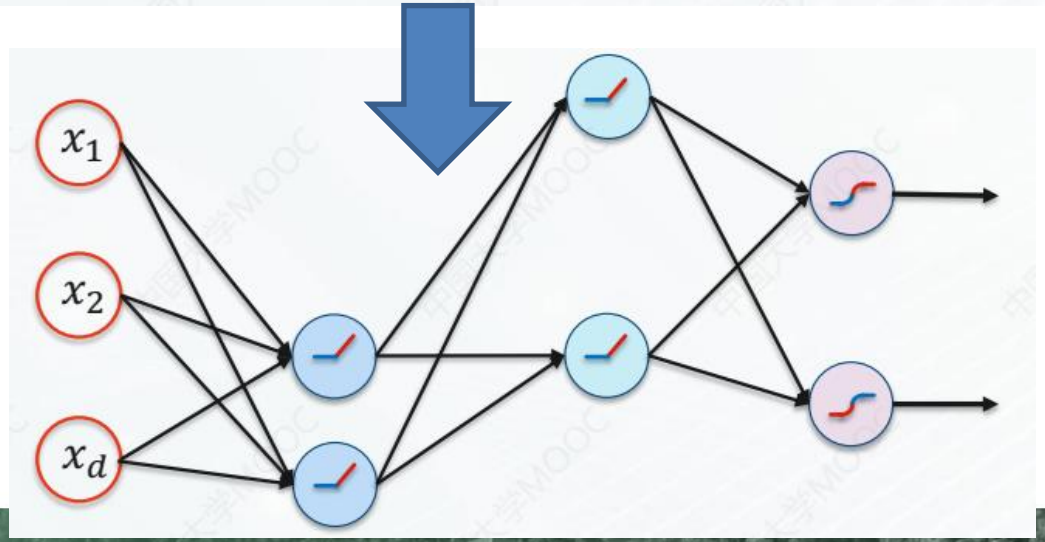
$$\frac{\partial f(x)}{\partial x}=\begin{cases}0, x<0\\ 1, x\geq 0\end{cases}$$

特点

- 计算速度快，收敛速度快，解决梯度消失问题
- 梯度为0，神经元输出为0；对网络输出没作用，无需更新
- 梯度为1，反向传播过程不衰减， 不出现梯度消失问题

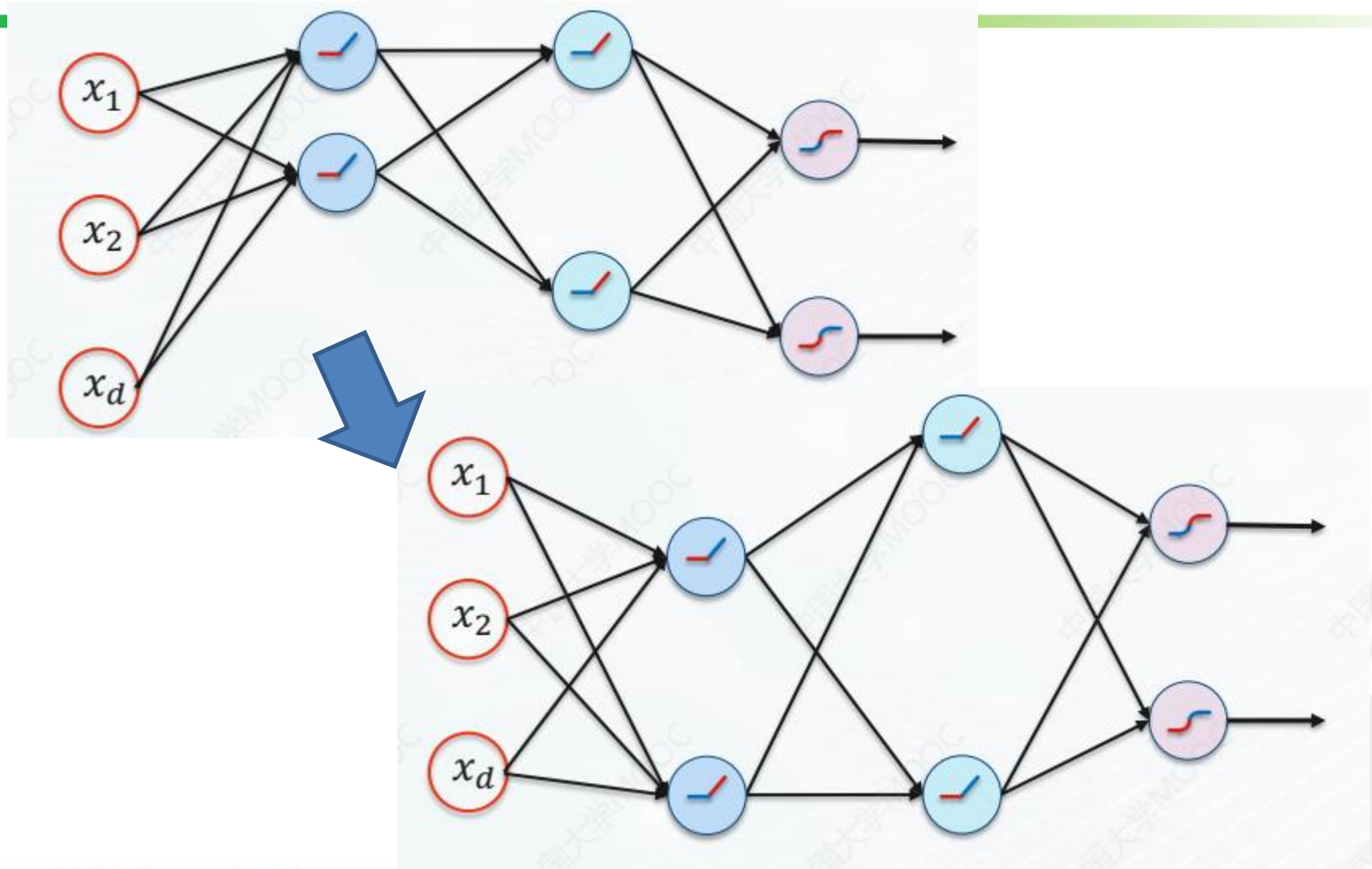
ReLu层的作用





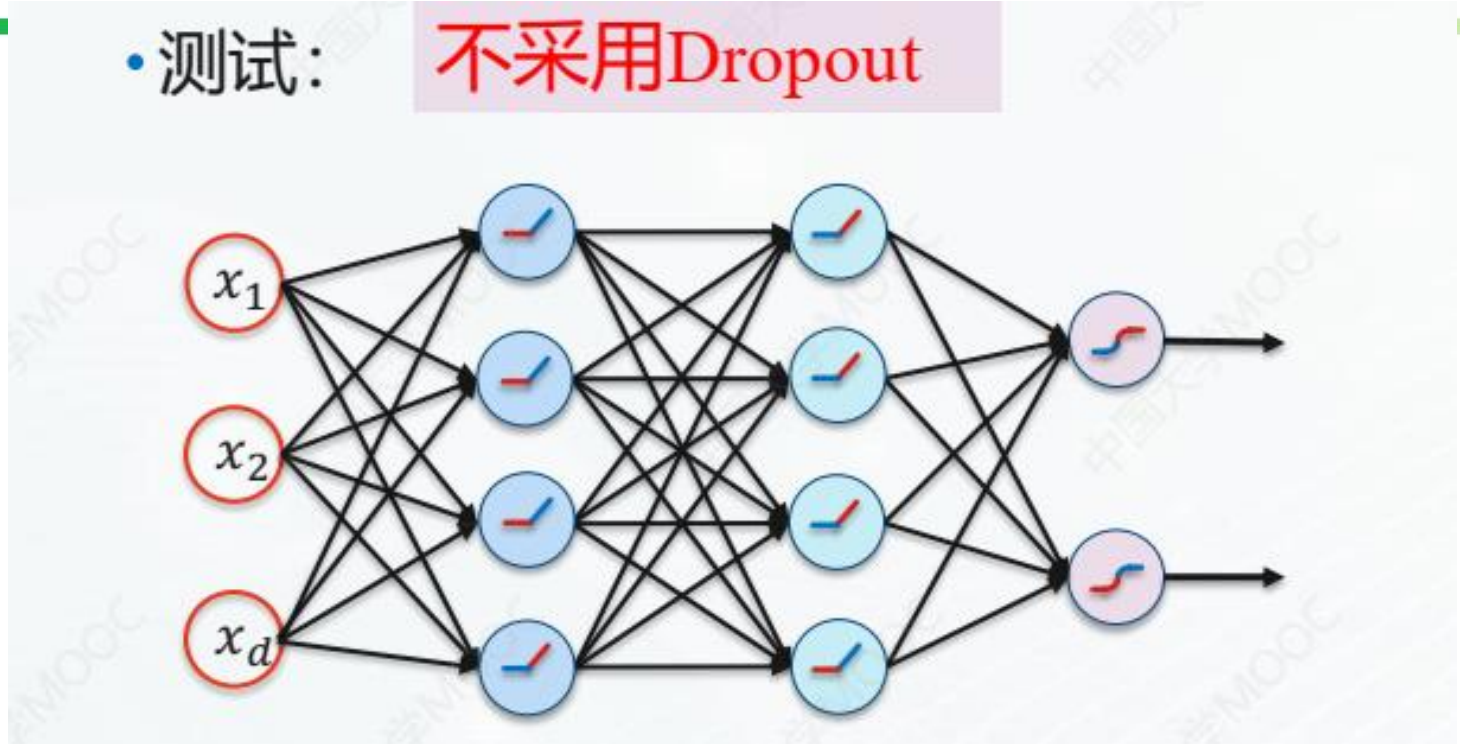
相当于简化了网络结构，减小了过拟合风险

Dropout层的作用：训练时



每批次用Dropout简化,再网络训练,下一训练批次，重新dropout

Dropout层的作用：测试时



测试时所有神经元的权重都需要乘上$(1 - p)$
使网络“瘦身”，减少过拟合风险
Dropout是一种集成学习，每次采样一个子网络
可以近似看作是集成了不同网络的模型组合

批标准化 BN(Batch Normalization层

对上一层网络输出z进行标准化处理，减少分布偏移

一般将输入 $z \in \Re d$ 归一化到标准正态分布

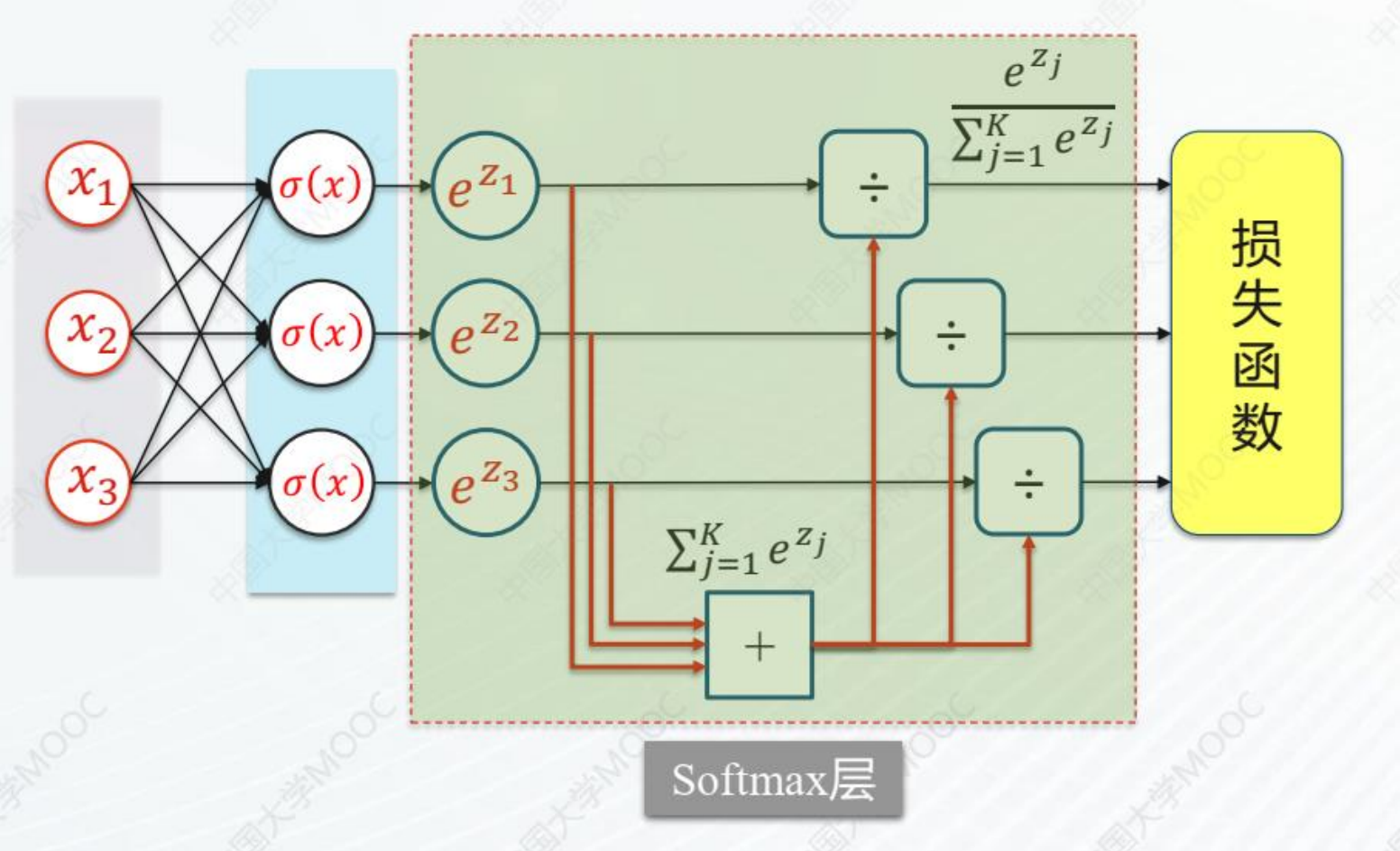
$$\hat{z} = \mathrm{BN}_{\gamma,\beta}(z) = \frac{z-\mu}{\sqrt{\sigma^2+\epsilon}} \odot \gamma + \beta$$

元素级乘法

BN一般加在每一层非线性激活函数之前， l表示层数

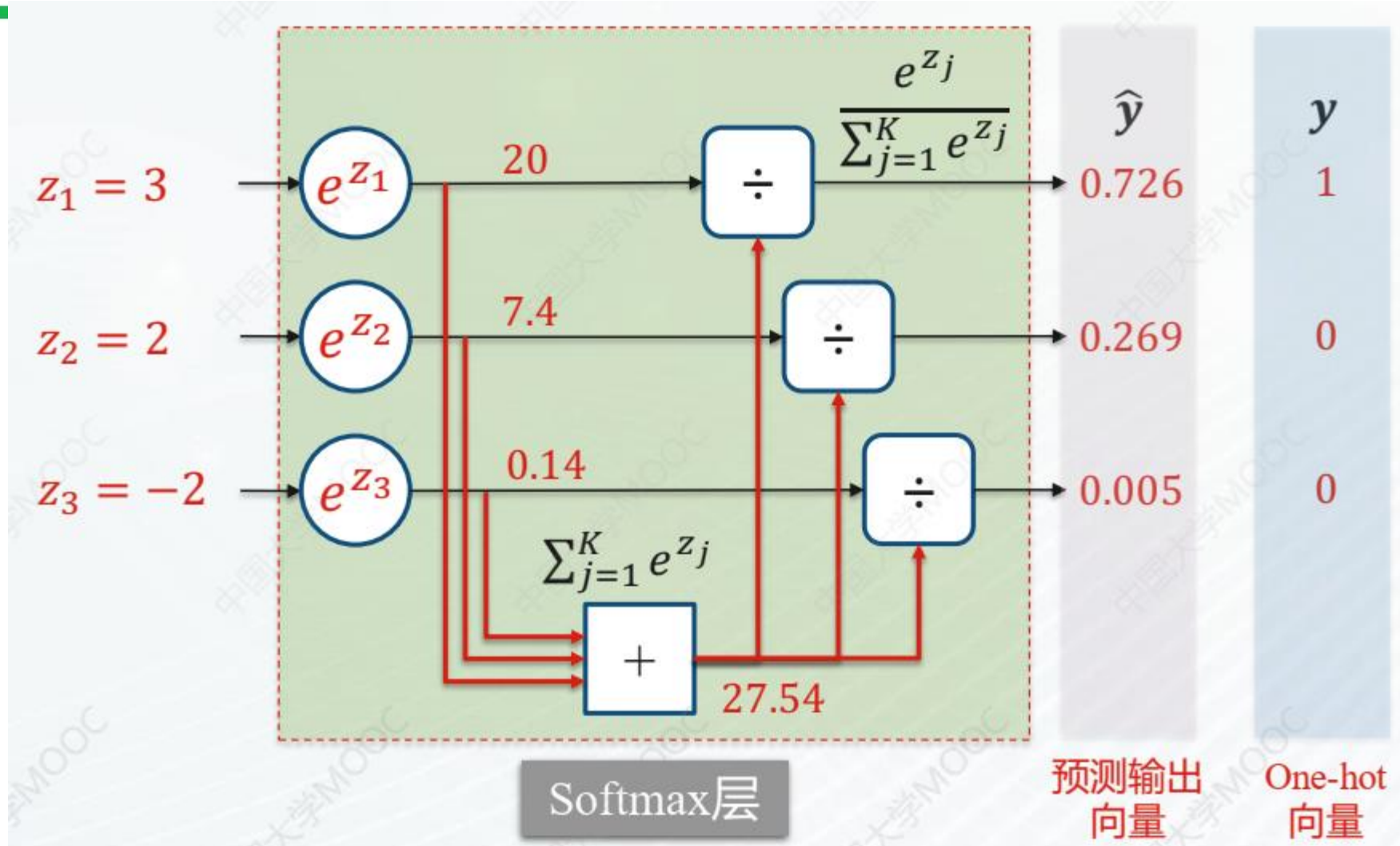
$$a^{(l)} = f\left(\mathrm{BN}_{\gamma,\beta}\left(z^{(l)}\right)\right) = f\left(\mathrm{BN}_{\gamma,\beta}\left(W a^{(l-1)}\right)\right)$$

Softmax 多分类层（激活函数）



$$f(x) = \text{softmax}(x) = \frac{\exp(x)}{\sum \exp(x)}$$

Softmax 层例子


$z_1 = 3$
$z_2 = 2$
$z_3 = -2$
e^{z_1}
e^{z_2}
e^{z_3}
20
7.4
0.14
$\sum_{j=1}^{K} e^{z_j}$
27.54
$\frac{e^{z_j}}{\sum_{j=1}^{K} e^{z_j}}$
$\hat{y}$
0.726
0.269
0.005
y
1
0
0
Softmax层
预测输出向量
One-hot向量

交叉熵损失函数（层）

函数形式

$$H(X) = -\sum_{i=1}^{N} p(\boldsymbol{x}_i)\log(q(\boldsymbol{x}_i))$$

$p(\boldsymbol{x}_i)$：真实概率分布　　$q(\boldsymbol{x}_i)$：预测概率分布

特点：

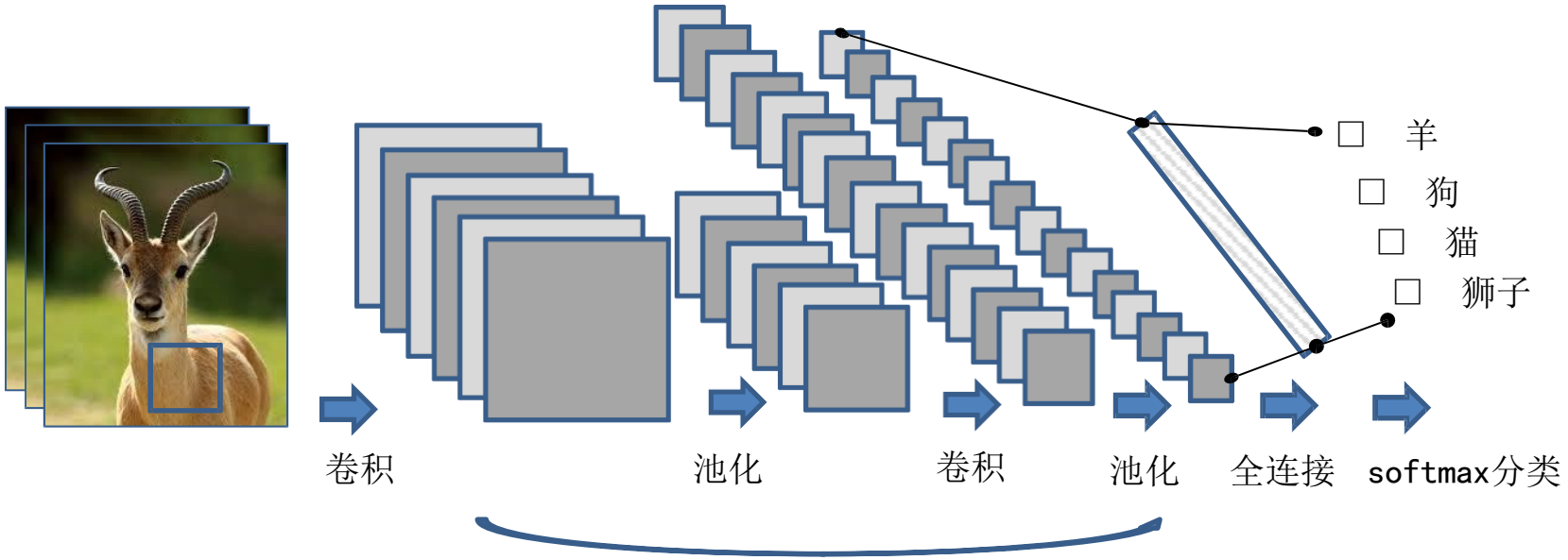
交叉熵越小，真实分类和预测分类的两个分布越接近

例子：

例如：$\hat{\boldsymbol{y}}_i = [0.726, 0.269, 0.005]^T; \boldsymbol{y}_i = [1,0,0]^T$

$$H(\boldsymbol{x}_i) = -[1,0,0] \times \log\left(\begin{bmatrix} 0.726 \\ 0.269 \\ 0.005 \end{bmatrix}\right) \approx -0.320$$

多层处理实现举例



- **一层卷积及降采样：往往只学到了局部的特征。**
- **多层卷积及降采样：层数越多，学到的特征越全局化**
  ➔通过这样的多层处理，低级的特征组合形成更高级的特征表示。

用Matlab构建深度学习



深度学习主要框架


Microsoft
CNTK
Caffe
Caffe2
PYTORCH
Chainer
Keras
TensorFlow
theano
∂y/net
mxnet
GLUON

深度学习主要框架1

名称	优点	缺点
TensorFlow	• Google开发维护， • 底层C++编写， • 前台支持Python等多语言 • 不完美但最流行 • 社区强大，适合生产环境。	• 过于复杂的系统设计 • 频繁变动的接口 • 接口设计晦涩难懂 • 文档混乱脱节 • 相对底层的框架
Keras	• 由纯Python编写， • 后端TensorFlow、Theano及CNTK • 入门最简单	• 深度学习接口，构建于第三方框架之上 • 但是不够灵活，使用受限。
Caffe/ Caffe2	• 性能优异，几乎全平台支持（Caffe2） • 配置文件设置模型 • 适合生产环境 • Facebook维护	• 文档不够完善 • 主要支持卷积模型 • 新层需要自编程 • 配置但不能编程

深度学习主要框架2

名称	优点	缺点
PyTorch	• 简洁设计 • 面向对象设计优雅 • 提供完整文档，渐进指南 • Facebook维护	• 没有Keras简洁 • 一些功能不易实现
CNTK	• 性能比Caffe、Theano、TensoFlow等主流工具都要强	• 最初是在 MS内部使用而开发的， • 文档难懂，推广不给力
MATLAB	• 代码简洁规范 • 易用性，容易入门 • 文档规范 • 可导入导出主流框架模型	• 性能不断提高 • 灵活性不断提高

Matlab构建深度学习模型的思路

问题	对策	GUI	编程
Matlab没有，但其他深度学习框架有预训练模型	1）某些框架可以直接导入 2）混合编程调用 3）导出导入ONNX 格式	√	√
Matlab有处理类似问题的预训练模型 ➔深度学习模型库	迁移学习+直接使用预训练模型	√	√
	迁移学习+用新数据再训练实现微调	√	√
从头构建新模型	模仿例子代码进行构建	√	√

其他深度学习框架： TensorFlow、PyTorch、Caffe2、Microsoft Cognitive Toolkit (CNTK)、Core ML 和 Apache MXNet™。

GUI：深度网络设计器 deepNetworkDesigner

中山 SUN YAT-SEN UNIVERSITY

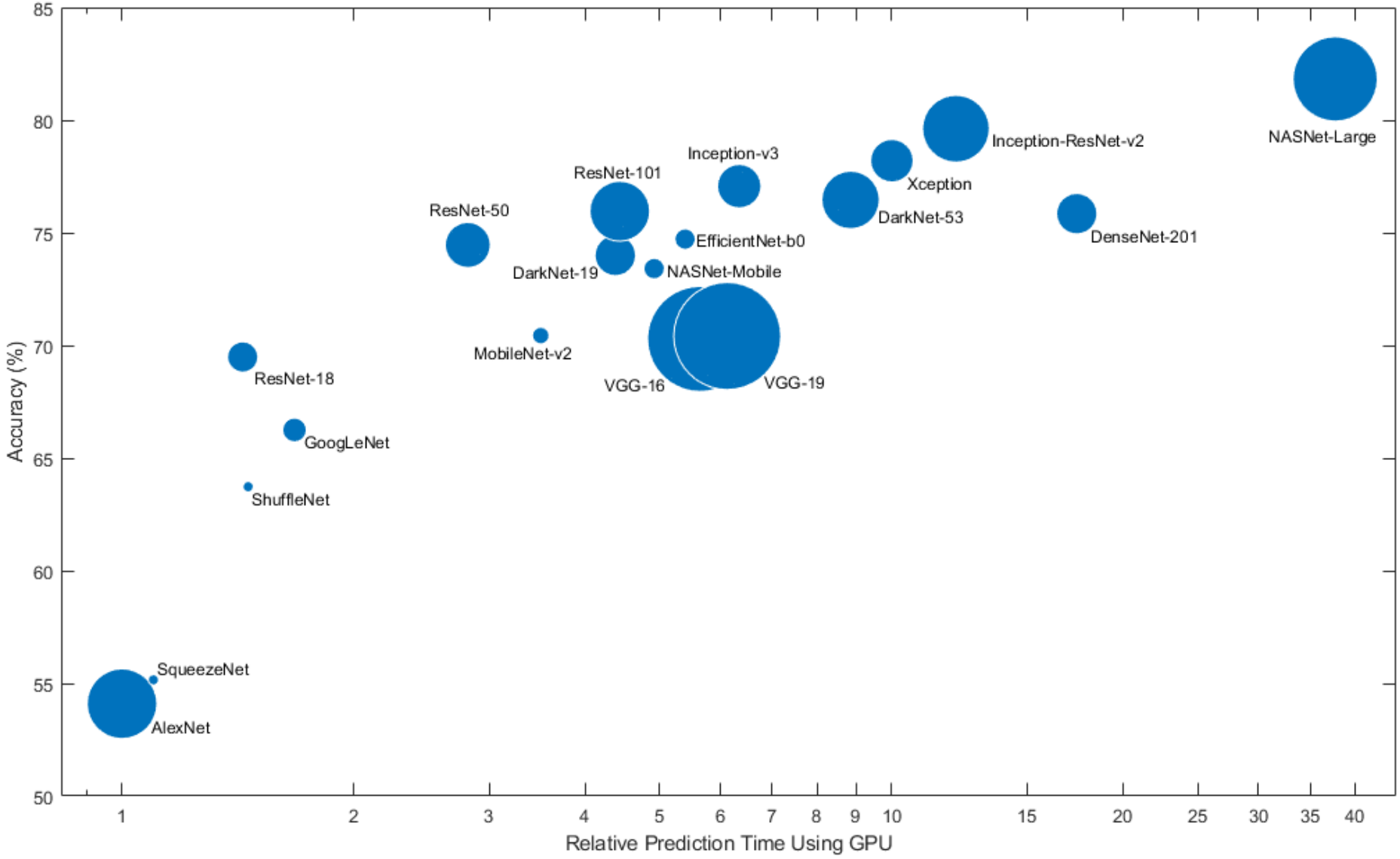
可以使用预训练模型的情况

考虑因素	创建新的深度网络	使用预训练网络进行迁移学习
训练数据	数千到数百万张已标注图像	数百到数千张已标注图像（小）
计算量	计算量大（需要使用 GPU 来提高速度）	中等计算量（可选择使用 GPU）
训练时间	解决真实问题需要几天到几周的时间	数秒到数分钟
模型准确度	高，但对于较小的数据集，可能会出现过拟合的情况	好，取决于预训练模型

常见的预训练模型

- 大多数预训练网络是基于 ImageNet 数据库 的子集进行训练的，该数据库用于 ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) 中。
- 这些网络已经对超过一百万个图像进行了训练，可以将图像分为 1000 个对象类别，例如键盘、咖啡杯、铅笔和多种动物。
- 通常来说，使用预训练网络进行迁移学习比从头开始训练网络更快更容易。

Matlab中预训练网络的速度比较



Matlab中预训练网络的结构比较1

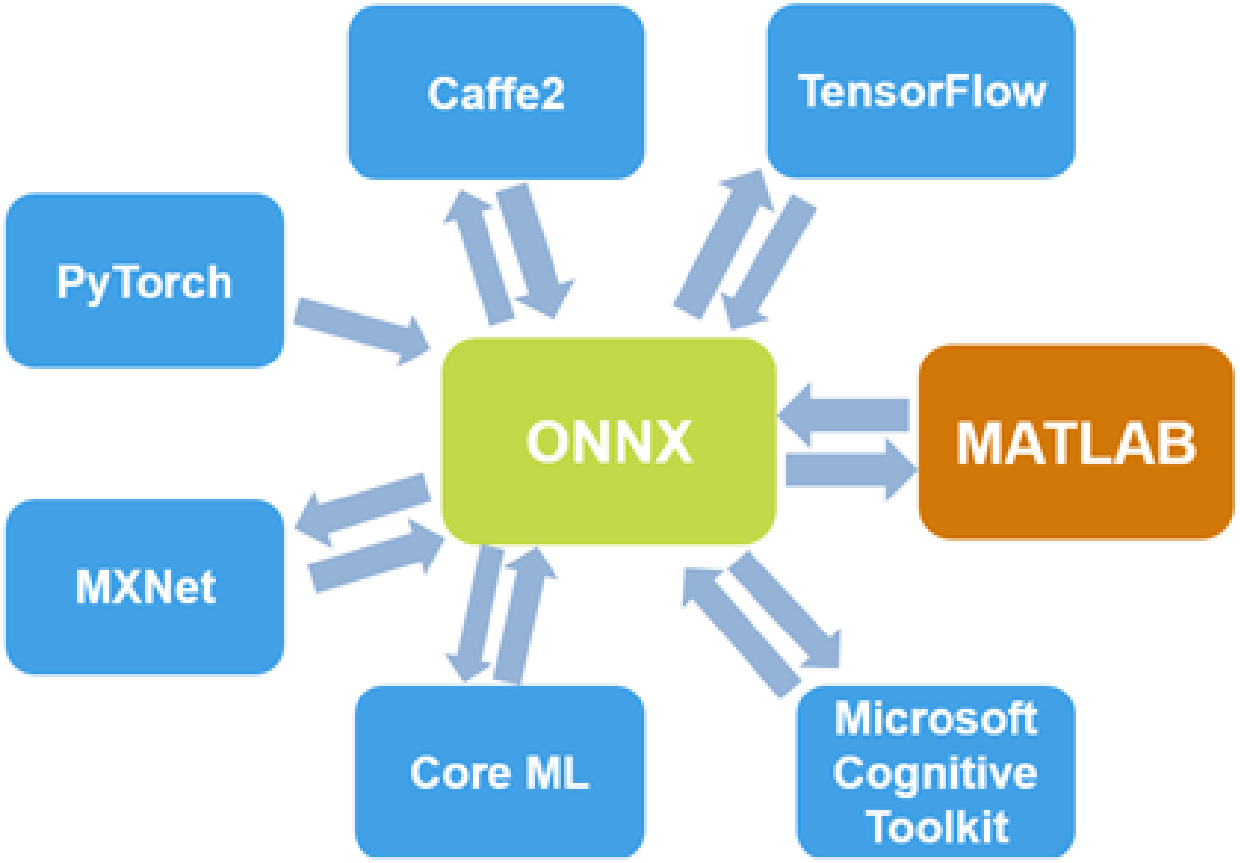
网络	深度	大小	参数（百万	图像输入
squeezenet	18	5.2 MB	1.24	227×227
googlenet	22	27 MB	7.0	224×224
inceptionv3	48	89 MB	23.9	299×299
densenet201	201	77 MB	20.0	224×224
mobilenetv2	53	13 MB	3.5	224×224
resnet18	18	44 MB	11.7	224×224
resnet50	50	96 MB	25.6	224×224
resnet101	101	167 MB	44.6	224×224
xception	71	85 MB	22.9	299×299
inceptionresnetv2	164	209 MB	55.9	299×299

Matlab中预训练网络的结构比较2

网络	深度	大小	参数（百万）	图像输入
shufflenet	50	5.4 MB	1.4	224×224
nasnetmobile	*	20 MB	5.3	224×224
nasnetlarge	*	332 MB	88.9	331×331
darknet19	19	78 MB	20.8	256×256
darknet53	53	155 MB	41.6	256×256
efficientnetb0	82	20 MB	5.3	224×224
alexnet	8	227 MB	61.0	227×227
vgg16	16	515 MB	138	224×224
vgg19	19	535 MB	144	224×224

Matlab中导入和导出深度学习网络

从 TensorFlow®-Keras、Caffe 和 ONNX™（开放式神经网络交换）模型格式导入网络和网络架构。您还可以将经过训练的网络导出为 ONNX 模型格式。



Matlab中导入和导出深度学习网络

	格式1	格式2
从Keras导入	使用 importKerasNetwork 从 TensorFlow-Keras 导入预训练网络。	使用 importKerasLayers 从 TensorFlow-Keras 导入网络架构(使用或不使用权重均可)。
从Caffe导入	使用 importCaffeNet-work 函数从 Caffe 中导入预训练网络。Caffe Model Zoo 中提供许多预训练网络。	下载所需的 .prototxt 文件，并使用 importCaffeLayers 将网络层导入 MATLAB。
在ONNX中导入和导出	使用 exportONNXNet-work 函数将经过训练的 Matlab网络导出为 ONNX 格式，再导入到其他框架中。	使用 importONNXNet-work 从 ONNX 导入预训练网络，并使用 importONNXLayers 导入带或不带权重的网络架构。

Matlab构建深度学习典型例子代码

任务： 输入数据+功能	例子代码
图像分类	创建简单的深度学习网络以用于分类 训练残差网络进行图像分类
图像回归	针对回归训练卷积神经网络
序列分类 时序回归	使用深度学习进行序列分类 使用深度学习进行时序预测
音频数据	使用深度学习进行语音命令识别
文本数据	使用深度学习对文本数据进行分类 使用深度学习生成文本

构建深度学习模型基本步骤

步骤	基本命令	功能
训练模型	layers = [imageInputLayer([28 28 1]) convolution2dLayer(5,20) reluLayer fullyConnectedLayer(10) softmaxLayer classificationLayer];	定义网络结构
	options = trainingOptions('adam');	定义全局训练参数
	trainedNet = trainNetwork(data,layers,options);	训练网络
预测	predict	回归预测
	classify	分类预测
评估	预测性能的评估	

SUN YAT-SEN UNIVERSITY

Layer的层函数：输入层

层	说明
imageInputLayer	图像输入层向网络输入**二维图像**，并应用数据归一化。
image3dInputLayer	**三维图像输入层**向网络输入**三维图像或三维体**，并应用数据归一化。
sequenceInputLayer	**序列**输入层向网络输入序列数据。
featureInputLayer	**特征**输入层将特征数据输入网络并应用数据归一化。当您有表示特征的数值标量数据集（**数据没有空间和时间维度**）时，请使用此层。
roiInputLayer (Computer Vision Toolbox)	**ROI 输入层**将图像输入到 Fast R-CNN 目标检测网络。

Layer的层函数：卷积和全连接层

层	说明
convolution2dLayer	**二维卷积层**对输入应用滑动卷积滤波器。
convolution3dLayer	**三维卷积层**将滑动立方体卷积滤波器应用于三维输入。
groupedConvolution2dLayer	二维分组卷积层将输入通道分成各个组，并应用滑动卷积滤波器。使用分组卷积层进行按通道可分离（也称为按深度可分离）卷积。
transposedConv2dLayer	转置的二维卷积层对特征图进行上采样。
transposedConv3dLayer	转置的三维卷积层对三维特征图进行上采样。
fullyConnectedLayer	**全连接层**将输入乘以权重矩阵，然后添加偏置向量。

Layer的层函数：序列层1

层	说明
sequenceInputLayer	**序列输入层**向网络输入序列数据。
lstmLayer	**LSTM 层**学习时序和序列数据中时间步之间的长期相关性。
bilstmLayer	**双向 LSTM (BiLSTM) 层**学习时序或序列数据的时间步之间的双向长期相关性。当您希望网络在每个时间步从完整时序中学习时，这些相关性会很有用。
gruLayer	**GRU 层**学习时序和序列数据中时间步之间的相关性。

Layer的层函数：序列层2

层	说明
sequenceFoldingLayer	**序列折叠层**将一批图像序列转换为一批图像。使用序列折叠层独立地对图像序列的时间步执行卷积运算。
sequenceUnfoldingLayer	**序列展开层**在序列折叠后还原输入数据的序列结构。
flattenLayer	**扁平化层**将输入的空间维度折叠成通道维度。
wordEmbeddingLayer (Text Analytics Toolbox)	**单词嵌入层**将单词索引映射到向量。

Layer的层函数：激活层

层	说明
reluLayer	**ReLU 层**对输入的每个元素执行阈值运算，其中任何小于零的值都设置为零。
leakyReluLayer	**泄漏 ReLU 层**执行阈值运算，其中小于零的任何输入值都乘以固定标量。
clippedReluLayer	**裁剪 ReLU 层**执行阈值运算，其中任何小于零的输入值都设置为零，*裁剪上限*以上的任何值都设置为该裁剪上限。
eluLayer	**ELU 激活层**对正输入执行单位运算，对负输入执行指数非线性运算。
tanhLayer	**双曲正切 (tanh) 激活层**对层输入应用 tanh 函数。
swishLayer	**swish 激活层**对层输入应用 swish 函数。
preluLayer（自定义层示例）	**PReLU 层**执行阈值运算，其中对于每个通道，任何小于零的输入值都乘以在训练时学习到的标量。

Layer的层函数：归一化

层	说明
batchNormalizationLayer	**批量归一化层**独立地对每个通道的所有观测值的小批量数据进行归一化。为了加快CNN的训练并降低对网络初始化的敏感度，请在卷积层和非线性部分之间使用批量归一化层，如 ReLU 层。
groupNormalizationLayer	**组归一化层**独立地对每个观测值的通道分组子集的小批量数据进行归一化。为了加快CNN的训练并降低对网络初始化的敏感度，请在卷积层和非线性部分之间使用组归一化层，如 ReLU 层。
instanceNormalizationLayer	**实例归一化层**独立地对每个观测值的每个通道上的小批量数据进行归一化。为了改进训练CNN的收敛性并降低对网络超参数的敏感度，请在卷积层和非线性部分之间使用实例归一化层，如 ReLU 层。

Layer层函数：归一化、丢弃层

层	说明
layerNormalizationLayer	**层归一化层**独立地对每个观测值的所有通道上的小批量数据进行归一化。为了加快循环和多层感知神经网络的训练并降低对网络初始化的敏感度，请在可学习层后使用层归一化层，例如 LSTM 层和全连接层。
crossChannelNormalizationLayer	按**通道局部响应（跨通道）归一化层**执行按通道归一化。
dropoutLayer	**丢弃层**以给定的概率将输入元素随机设置为零。
crop2dLayer	**二维裁剪层**对输入应用二维裁剪。
crop3dLayer	**三维裁剪层**将三维体裁剪到输入特征图的大小。

Layer层函数：裁剪层

层	说明
resize2dLayer (Image Processing Toolbox)	**二维调整大小层**根据缩放因子将二维输入的大小调整到指定的高度和宽度，或调整到引用输入特征图的大小。
resize3dLayer (Image Processing Toolbox)	**三维调整大小层**根据缩放因子将三维输入的大小调整到指定的高度、宽度和深度，或调整到引用输入特征图的大小。

Layer层函数：平均池化和去池化层

层	说明
averagePooling2dLayer	**平均池化层**通过将输入划分为矩形池化区域并计算每个区域的平均值来执行下采样。
averagePooling3dLayer	**三维平均池化层**通过将三维输入划分为立方体池化区域并计算每个区域的平均值来执行下采样。
globalAveragePooling2dLayer	**全局平均池化层**通过计算输入的高度和宽度维度的均值来执行下采样。
globalAveragePooling3dLayer	**三维全局平均池化层**通过计算输入的高度、宽度和深度维度的均值来执行下采样。

Layer层函数：最大池化和去池化层

层	说明
maxPooling2dLayer	**最大池化层**通过将输入划分为矩形池化区域并计算每个区域的最大值来执行下采样。
maxPooling3dLayer	**三维最大池化层**通过将三维输入划分为立方体池化区域并计算每个区域的最大值来执行下采样。
globalMaxPooling2dLayer	**全局最大池化层**通过计算输入的高度和宽度维度的最大值来执行下采样。
globalMaxPooling3dLayer	**三维全局最大池化层**通过计算输入的高度、宽度和深度维度的最大值来执行下采样。
maxUnpooling2dLayer	**最大去池化层**对最大池化层的输出进行去池化。

Layer层函数：组合层

层	说明
additionLayer	**相加层**按元素将来自多个神经网络层的输入相加。
multiplicationLayer	**相乘层**按元素将来自多个神经网络层的输入相乘。
depthConcatenationLayer	**深度串联层**接受具有相同高度和宽度的输入，并沿第三个维度（通道维度）串联它们。
concatenationLayer	**串联层**接受输入并沿指定维度串联它们。除串联维度外，所有其他维度中的输入必须具有相同的大小。
weightedAdditionLayer（自定义层示例）	**加权相加层**按元素对来自多个神经网络层的输入进行缩放和相加。

Layer层函数：输出层

层	说明
softmaxLayer	**softmax 层**对输入应用 softmax 函数。
sigmoidLayer	**sigmoid 层**对输入应用 sigmoid 函数，使得输出在区间 (0,1) 内有界。
classificationLayer	**分类层**计算具有互斥类的分类和加权分类任务的交叉熵损失。
regressionLayer	**回归层**计算回归任务的半均方误差损失。
sseClassificationLayer（自定义层示例）	**分类 SSE 层**计算分类问题的误差损失平方和。
maeRegressionLayer（自定义层示例）	**回归 MAE 层**计算回归问题的均值绝对误差损失。

其他层介绍：https://ww2.mathworks.cn/help/deeplearning/ug/list-of-deep-learning-layers.html

trainingOptions函数的调用格式

格式	用途
options = trainingOptions(solverName)	返回由solverName指定的优化器的训练选项。要训练网络，请使用训练选项作为trainNetwork函数的输入参数。
options = trainingOptions(solverName,Name,Value)	返回带有由一个或多个名称-值对参数指定的附加选项的训练选项。
options = trainingOptions('sgdm', ... 'LearnRateSchedule','piecewise', ... 'LearnRateDropFactor',0.2, ... 'LearnRateDropPeriod',5, ... 'MaxEpochs',20, ... 'MiniBatchSize',64, ... 'Plots','training-progress')	创建一组选项，来训练一个网络使用带动量的随机梯度下降法求解。每隔5个周期将学习速率降低0.2倍。将训练的最大时间间隔设置为20，并在每次迭代中使用带有64个观察值的小批处理，打开训练进度图。

trainingOptions的solverName格式

求解器	方法	特点	备注
sgdm	带动量的随机梯度下降法	加权平均之后梯度进行参数更新，含有之前梯度的信息（动量），可以加速收敛，减少振荡	用“momentum”名称-值对参数指定动量值。
rmsprop	均方根传播法	可以消除梯度下降中的摆动，包括mini-batch梯度下降，并允许使用一个更大的学习率，从而加快收敛	用SquaredGradient Decay Factor名称-值对参数，指定平方梯度移动平均的衰减率。
adam	自适应矩估计法	结合了上面两个方法	用'GradientDecay Factor’和'Squared GradientDecayFactor'名称-值对参数，指定梯度和平方梯度移动平均的衰减率。

https://ww2.mathworks.cn/help/deeplearning/ref/trainingoptions.html

trainNetwork函数的调用格式

格式	用途
net = trainNetwork(images,layers,options)	使用images指定的图像和响应，options定义的训练选项，训练由layers指定的网络进行分类和回归。
trainNetwork(images,responses,layers,options)	训练使用images指定的图像和responses指定的响应。
trainNetwork(sequences,layers,options)	使用sequences指定的序列和响应，训练用于序列或时间序列分类和回归任务的网络(如LSTM或GRU)。
trainNetwork(sequences,responses,layers,options)	使用sequences指定的序列和responses指定的响应来训练。
trainNetwork(features,layers,options	使用responses指定的特征及其响应训练网络进行分类或回归。
trainNetwork(features,responses,layers,options)	使用features指定的特征和responses指定的响应来训练。
[net,info] = trainNetwork(___)	使用前面任何语法，返回训练信息

predict函数的调用格式：回归

格式	用途
YPred = predict(net,imds)	使用训练的SeriesNetwork或DAGNet-work对象网络预测imds中图像数据的响应。
YPred = predict(net,ds)	预测数据存储ds中数据的响应。
YPred = predict(net,tbl)	预测表tbl中数据的响应。
YPred = predict(net,X)	预测对数组X中的图像或特征数据的响应。
YPred = predict(net,X1，…，XN)	预测多输入网络中数字数组X1，…，XN中的数据的响应。输入Xi对应网络输入net.InputNames(i)。
[YPred1,…，YPredM] = predict(___)	使用前面任何语法预测多输出网络的M个输出的响应。输出YPredj对应于网络输出net.OutputNames(j)。
YPred = predict(net,sequences)	使用经过训练的递归网络(如LSTM或GRU网络)预测序列中的序列或时间序列数据的响应。
___ = predict(___，Name,Value)	用一个或多个名称-值对参数指定的附加选项预测响应。

classify函数的调用格式：分类

格式	用途
YPred = classify (net,imds)	使用训练好的网络net预测图像数据存储imds中图像的类别标签。
YPred = classify (net,ds)	预测数据存储ds中的数据的类标签。
YPred = classify (net,X)	预测由数组X指定图像或特征数据的类标签。
YPred = classify (net,X1，…，XN)	预测多输入网络net中的数字数组X1，…，XN中的数据的类标签。输入Xi对应网络输入net.InputNames(i)。
YPred = classify (net,sequences)	为递归网络(例如LSTM或GRU网络)网络中的时间序列或序列数据预测分类标签。
YPred = classify (net,tbl)	预测表tbl中数据的类标签。
YPred = classify (___，Name,Value)	预测类标签的附加选项，选项由一个或多个名称-值对参数，使用任何前面的语法指定。
[YPred,scores] = classify (___)	返回与使用前面任何语法的类标签对应的分类分数。

5. 小结

思考?

- 什么是深度学习?
- LSTM的结构特点及输入数据?
- CNN的结构特点及输入数据?
- 怎么设计深度学习的结构?
- 怎么优化深度学习的结构?
- 深度学习如何求解?
- 实现深度学习的常用框架?
- 深度学习用Matlab如何实现?

推荐的慕课

- 机器学习（浙江大学）
- https://www.icourse163.org/learn/ZJU-1206573810
- 深度学习基础（哈尔滨工业大学）
- https://www.icourse163.org/learn/HIT-1206320802
- 深度学习应用开发-TensorFlow实践
- https://www.icourse163.org/learn/ZUCC-1206146808